



دانشگاه شهید بهشتی
دانشکده مهندسی و علوم کامپیوتر

طراحی سامانه‌ی خودکار سازی تجهیزات تست صنعتی

پروژه کارشناسی
معماری کامپیوتر

دانشجو:
محمد معین عربی

استاد راهنما:
دکتر سیدحسین عطارزاده نیاکی

تابستان ۱۴۰۴

چکیده

با پیشرفت صنایع الکترونیکی، خودکارسازی فرایندهای اندازه‌گیری به یکی از چالش‌های مهم برای تضمین کیفیت، سرعت و دقت تبدیل شده است. تا کنون ابزارهای متن‌باز و تجاری متنوعی با قابلیت‌های منحصربه‌فرد برای حل این چالش توسعه یافته‌اند. نرم‌افزارهای تجاری متن‌بسته، ماهیت افزودن قابلیت‌های دلخواه متناسب با نیاز را ندارند و در پشتیبانی از سخت‌افزارها و پروتکل‌های ارتباطی متنوع محدود هستند. از طرفی، پروژه‌های متن‌باز موجود نیز فاقد یکپارچگی برای داشتن همه‌ی نیازمندی‌ها شامل کنترل از راه دور تجهیزات، خودکارسازی اندازه‌گیری و داشتن یک رابط کاربری گرافیکی هستند. برای پاسخ به این نیاز، این پروژه یک چارچوب نرم‌افزاری بر پایه‌ی پایتون، با معماری توزیع‌شده و لایه‌ای ارائه می‌دهد که شامل یک رابط کاربری گرافیکی تحت وب برای تعامل کاربر و پایش لحظه‌ای تجهیزات، یک موتور اجرای آزمون (بر پایه‌ی Robot Framework) برای تعریف و خودکارسازی اندازه‌گیری، و لایه‌های انتزاعی نرم‌افزاری برای ارتباط و کنترل تجهیزات است. همچنین یک مبدل سخت‌افزاری برای تبدیل پروتکل GPIB و CAN با استفاده از Raspberry Pi و درایورهای متن‌باز لینوکس پیاده‌سازی شد که دسترسی به تجهیزات سخت‌افزاری را تحت شبکه به اشتراک می‌گذارد و به داشتن یک معماری توزیع‌شده کمک می‌کند.

نتیجه نهایی، یک سامانه آزمون خودکار است که فرایند تعریف اندازه‌گیری، کنترل از راه دور تجهیزات، اجرای آزمون‌ها و پایش لحظه‌ای نتایج را فراهم می‌کند.

واژگان کلیدی: آزمون خودکار، تجهیزات اندازه‌گیری، پایتون، GPIB، متن‌باز، Robot Framework

فهرست مطالب

۱	فصل اول: کلیات	۱
۳	مقدمه	۱-۱
۳	بیان مسئله	۱-۲
۴	کلیات روش پیشنهادی	۱-۳
۴	هدف پروژه	۱-۳-۱
۵	معرفی روش پیشنهادی	۱-۳-۲
۵	مراحل اجرای پروژه	۱-۳-۳
۷	روندنمای کلی پروژه	۱-۳-۴
۷	ساختار پروژه	۱-۴
۸		۲
۹	مقدمه	۲-۱
۹	کلید واژه‌ها	۲-۱-۱
۱۲	پروتکل GPIB	۲-۱-۲
۱۷	مبدل GPIB-Ethernet شرکت Prologix	۲-۱-۳
	دستورات استاندارد برای تجهیزات قابل برنامه‌ریزی (SCPI)	۲-۱-۴
۲۰		
۲۲	مطالعات انجام شده روی تجهیزات	۲-۱-۵
۲۸	تحلیل نقاط قوت و ضعف منابع غیرپژوهشی مشابه	۲-۲
۲۸	ابزارهای کنترل تجهیزات	۲-۲-۱
۳۱	ابزارهای اجرای آزمون و خودکارسازی اندازه‌گیری	۲-۲-۲
۴۱	ابزارهای رابط گرافیکی	۲-۲-۳
۴۲	ابزارهای ارتباطی	۲-۲-۴
۴۲	ابزارهای شبیه‌سازی	۲-۲-۵
۴۳	جمع‌بندی	۲-۳
۴۴		۳

۴۵	مقدمه	۳-۱
۴۵	ساختار روش پیشنهادی	3-2
۴۵	گردش کار کلی سیستم	۳-۲-۱
۴۷	معماری سطح بالای سیستم	۳-۲-۲
۴۹	پیاده‌سازی روش پیشنهادی	۳-۳
۴۹	سطح ۲: Container Diagram	3-3-1
۵۲	سطح ۳ و ۴: Component/code Diagram	3-3-2
۶۲	روش ارزیابی	3-4
۶۵	تاخیر	۳-۴-۱
۶۶	صحت‌سنجی اندازه‌گیری	۳-۴-۲
۶۷	مجوزها	3-4-3
۶۸	نتایج	۳-۵
۶۸	جمع‌بندی	۳-۶

فهرست شکل‌ها

- شکل ۱ روندنمای مراحل کلی پیاده‌سازی ۷
- شکل ۲ نمای روبه‌روی دستگاه کانتر در حال نمایش یک اندازه‌گیری فرکانس ۹
- شکل ۳ نمای روبه‌روی دستگاه مولتی‌متر مدل HP3458A که در این پروژه مورد استفاده قرار گرفت ۱۰
- شکل ۴ نمای پشت دستگاه شمارنده دارای پورت GPIB ۱۰
- شکل ۵ نمای پین‌های کانکتور پورت GPIB ۱۲
- شکل ۶ فرآیند ارسال یک دستور به دستگاه GPIB ۱۴
- شکل ۷ نمودار تغییرات سیگنال‌های Handshake باس GPIB بر حسب زمان ۱۶
- شکل ۸ نمای جلو و عقب مبدل Prologix ۱۷
- شکل ۹ ترتیب فرآیند ارسال یک دستور و دریافت پاسخ به کمک مبدل Prologix ۱۹
- شکل ۱۰ ساختار درختی دستورات SCPI ۲۰
- شکل ۱۱ نمای روبه‌روی دستگاه تولیدکننده سیگنال ۲۶
- شکل ۱۲ نمای پشت دستگاه تولیدکننده سیگنال که از طریق پورت USB قابل کنترل از راه دور است ۲۷
- شکل ۱۳ نمای روبه‌روی دستگاه اندازه‌گیر توان ۲۷
- شکل ۱۴ محیط برنامه Pymasure به صورت گرافیکی در حال نمایش اندازه‌گیری‌ها است ۳۰
- شکل ۱۵ تصویری از محیط برنامه TestStand ۳۳
- شکل ۱۶ تصویری از محیط گرافیکی اجرای فازها ۳۵
- شکل ۱۷ تصویری از رابط گرافیکی پیشرفته‌تر با نام TofuPilot ۳۶
- شکل ۱۸ محیط برنامه Flojoy و یک نمونه‌سازی برای اندازه‌گیری فرکانس به کمک بخش Flowchart ۳۸
- شکل ۱۹ تصویری از محیط برنامه توسعه VSCode که افزونه Robot نصب گردیده است و در حال نمایش یک فایل تست است ۴۱

شکل ۲۰ نمودار فعالیت بخش‌های مختلف سیستم از زمان تعریف سناریو آزمون	۵۰
شکل ۲۱ سطح Context معماری سیستم با رویکرد مدل C4	۵۲
شکل ۲۲ سطح Container معماری سیستم با رویکرد مدل C4	۵۴
شکل ۲۳ سطح Component معماری سیستم با رویکرد مدل C4	۵۷
شکل ۲۴ عکس از نمونه‌سازی اولیه محیط رابط گرافیکی برای انتخاب و اجرای فایل تست	۵۸
شکل ۲۵ صفحه پایش لحظه‌ای اندازه‌گیری‌ها و رویدادها حین اجرای آزمون	۵۹
شکل ۲۶ صفحه پایش اندازه‌گیری ساخته شده مبتنی بر فناوری React	۵۹
شکل ۲۷ صفحه محیط گرافیکی برای اتصال دستی به یک دستگاه و انجام یک اندازه‌گیری	۶۰
شکل ۲۸ نمای کد لایه انتزاعی سخت‌افزار	۶۱
شکل ۲۹ نمودار کلاس لایه کنترل سخت‌افزار و لایه موتور تست	۶۲
شکل ۳۰ نمودار کلاس ارتباط هسته چارچوب Robot با کلاس‌ها containerها	۶۳
شکل ۳۱ عکس مبدل GPIB ساخته شده به کمک بورد رزبری پای	۶۵
شکل ۳۲ عکس اتصال مبدل رزبری پای به دستگاه شمارنده	۶۶
شکل ۳۳ عکس مایکروکنترلر mcp2515 که برای تبدیل پین سریال SPI رزبری پای به پروتکل CAN مورد استفاده قرار گرفته است	۶۷
شکل ۳۴ محیط گرافیکی داشبورد تحلیل آماری Robot	۶۸
شکل ۳۵ گزارش تولید شده پس از اجرای یک نمونه تست	۷۱

فهرست جدول‌ها

جدول ۱ سیگنال‌های Handshake در GPIB	۱۳
جدول ۲ سیگنال‌های کنترل و مدیریت باس GPIB	۱۳
جدول ۳ برخی از دستورات مبدل Prologix	۱۸
جدول ۴ دستورات مشترک و استاندارد SCPI	۲۲
جدول ۵ مقایسه ابزارهای کنترل سخت‌افزار	۳۰
جدول ۶ مقایسه ابزارهای اجرای آزمون	۳۲
جدول ۷ مقایسه تاخیرهای ارسال دستور IDN* به دستگاه‌ها از لایه‌های مختلف	۶۹
جدول ۸ لیست کتابخانه‌ها و مجوزهای هر یک از کتابخانه‌ها	۷۱

فهرست کلمات اختصاری

Abbreviations

Pages numbers

DUT: Device Under Test	11
GPIB: General Purpose Interface Bus	5
SCPI: Standard Commands for Programmable Instruments	20
SPI: Serial Peripheral Interface	62

فصل اول: کلیّات

۱-۱ مقدمه

در صنایع پیشرفته امروزی، آزمون و اندازه‌گیری دقیق، بخش جدایی‌ناپذیر و حیاتی از فرآیندهای کنترل کیفیت و تحقیق و توسعه در تولید قطعات الکترونیکی و تجهیزات به شمار می‌رود. برای اطمینان از عملکرد صحیح و قابل اطمینان محصولات، لازم است آزمون‌های پیچیده و تکرارشونده بر روی آن‌ها اجرا شود. خودکارسازی این فرآیندها نه تنها سرعت و حجم تولید را به شکل چشمگیری افزایش می‌دهد، بلکه با حذف خطای انسانی، دقت و تکرارپذیری نتایج را نیز تضمین می‌کند. هسته اصلی این سامانه‌های آزمون خودکار، نرم‌افزاری است که وظیفه کنترل و برقراری ارتباط از راه دور^۱ با تجهیزات اندازه‌گیری مانند مولتی‌مترهای^۲ دیجیتال، شمارنده‌ها^۳ و منابع تغذیه^۴ را بر عهده دارد.

با این حال، بازار ابزارهای آزمون ارتباط از راه دور عمدتاً در انحصار محصولات تجاری و گران‌قیمت بوده است. این ابزارها اغلب با چالش‌هایی همچون وابستگی به سخت‌افزارهای یک شرکت خاص، یادگیری و هزینه‌های بالای مجوز^۵ همراه هستند. علاوه بر این، ماهیت متن‌بسته و انحصار آن‌ها بر روی یک سیستم‌عامل (ویندوز)، انعطاف‌پذیری در توسعه و یکپارچه‌سازی را محدود می‌سازد. در مقابل، رشد زبان‌های برنامه‌نویسی سطح بالا مانند پایتون و توسعه‌ی پروژه‌های متن‌باز، فرصت خوبی برای ایجاد جایگزین‌های انعطاف‌پذیرتر، مقرون‌به‌صرفه‌تر و متناسب با نیازمندی‌های خاص فراهم کرده است.

هدف اصلی این پروژه، طراحی و پیاده‌سازی یک چارچوب نرم‌افزاری متن‌باز بر پایه‌ی پایتون برای خودکارسازی فرآیندهای آزمون و اندازه‌گیری است. این سامانه با ارائه دو جزء کلیدی، کنترل و پایش تجهیزات اندازه‌گیری از راه دور و اجرای توالی^۶ ارسال دستورات اندازه‌گیری به تجهیزات، به دنبال ارائه راهکاری ماژولار، توسعه‌پذیر و کاربرپسند است.

۱-۲ بیان مسئله

فرآیند اندازه‌گیری و اعتبارسنجی نیازمند تعامل همزمان مهندسان آزمون با مجموعه‌ای متنوع از دستگاه‌های اندازه‌گیری

¹ Remote access

² Multimeter

³ Counter

⁴ Power supply

⁵ License

⁶ Sequence

فصل اول: کلیات

است. مشکل اصلی از آنجا ناشی می‌شود که راه‌اندازی یک سناریوی اندازه‌گیری، نیازمند پیکربندی دستی و هماهنگ‌سازی چندین دستگاه به صورت فیزیکی از طریق صفحه جلویی هر دستگاه است. این فرآیند نه تنها زمان‌بر و مستعد خطای انسانی است، بلکه اجرای مکرر آن با شرایط یکسان را تقریباً غیرممکن می‌سازد.

برای پاسخ به این نیاز، شرکت‌هایی مانند National Instruments و Keysight راهکارهای نرم‌افزاری تجاری توسعه داده‌اند. با این حال، این ابزارها با وجود قابلیت‌های گسترده، محدودیت‌های متن‌بسته بودن را دارند و این شرکت‌ها خود تولیدکننده‌ی تجهیزات اندازه‌گیری هستند و در سطح پشتیبانی از تجهیزات تولیدشده توسط دیگر شرکت‌ها و پروتکل‌های دیگر دچار ضعف هستند.

در سوی دیگر، جامعه متن‌باز کتابخانه‌های نرم‌افزاری متعددی ارائه کرده‌اند ولی درایورهای کنترل از راه دور موجود برای بسیاری از مدل‌های تجهیزات، ناقص و یا تست‌نشده هستند و تمام قابلیت‌های دستگاه را پوشش نمی‌دهند. این کتابخانه‌ها فاقد یک محیط یکپارچه برای تعریف اندازه‌گیری‌های متوالی، گزارش‌گیری نتایج و یک محیط گرافیکی هستند.

مسئله اصلی، نبود یک چارچوب نرم‌افزاری یکپارچه و توسعه‌پذیر است که برای تجهیزات مورد استفاده‌ی ما، درایورهای کامل کنترل از راه دور را فراهم کند و امکان تعریف و اجرای اندازه‌گیری‌های متوالی را داشته باشد. هدف، ارائه راهکاری است که مهندس آزمون ضمن پیکربندی و تنظیم اندازه‌گیری‌های مختلف با یک زبان اسکرپتی، از یک محیط گرافیکی پایش^۷ لحظه‌ای تجهیزات اندازه‌گیری بهره‌مند شود و با معماری توزیع‌شده خود، امکان اجرای اجزای مختلف سیستم (رابط کاربری، موتور تست و کنترل‌کننده سخت‌افزار) بر روی ماشین‌های مجزا در بستر شبکه را نیز ممکن می‌سازد.

۱-۳ کلیات روش پیشنهادی

۱-۳-۱ هدف پروژه

در حالت سنتی و غیر خودکار، مهندس آزمون باید برای انجام یک اندازه‌گیری، هر یک از دستگاه‌ها را به صورت دستی تنظیم و مقدار اندازه‌گیری شده را یادداشت کند. هدف اصلی این پروژه، طراحی و پیاده‌سازی نرم‌افزاری مبتنی بر پایتون برای

⁷ Monitor

فصل اول: کلیات

خودکارسازی این فرآیندهای اندازه‌گیری است تا مهندس آزمون با نوشتن یک اسکریپت، تجهیزات را پیکربندی و آماده اندازه‌گیری کند و از طریق یک محیط گرافیکی روند اجرای اسکریپت‌ها و اندازه‌گیری‌های انجام شده را پایش کند.

۲-۳-۱ معرفی روش پیشنهادی

برای دستیابی به این هدف، یک معماری چهارلایه پیشنهاد و پیاده‌سازی شده است که هر لایه وظایف مشخصی را بر عهده دارد:

- **لایه مبدل پروتکل^۸:** این لایه فیزیکی‌ترین بخش نرم‌افزار است که ارتباط با باس GPIB و CAN را مدیریت می‌کند. فرمان‌ها را از لایه بالاتر دریافت می‌کند و برای دستگاه ارسال می‌کند. در این پروژه، یک مبدل سخت‌افزاری با استفاده از یک برد رزبری‌پای و درایورهای متن‌باز پیاده‌سازی شد. همچنین این لایه تحت شبکه برای ارسال و دریافت داده‌ها کار می‌کند.
- **لایه کنترل سخت‌افزار^۹:** این لایه بر روی لایه مبدل پروتکل GPIB و CAN قرار می‌گیرد و مسئول ارتباط منطقی با تجهیزات اندازه‌گیری است. این بخش شامل درایورهای سطح بالا برای هر دستگاه است که دستورات پیچیده هر یک از دستگاه‌ها را به توابع ساده و قابل فهم (مانند `measure_voltage`) تبدیل می‌کند.
- **موتور اجرای آزمون^{۱۰}:** قلب سیستم که وظیفه تحلیل و اجرای سناریوهای آزمون تعریف‌شده توسط کاربر را بر عهده دارد. در این پروژه از چارچوب^{۱۱} Robot Framework به عنوان موتور اجرای آزمون استفاده شده است.
- **لایه رابط کاربری:** یک محیط گرافیکی مبتنی بر وب که به کاربر اجازه می‌دهد یک فایل سناریو آزمون را برای اجرا انتخاب کند، وضعیت دستگاه‌ها، اندازه‌گیری‌ها و نتایج را به صورت لحظه‌ای مشاهده کند.

۳-۳-۱ مراحل اجرای پروژه

فرآیند طراحی و پیاده‌سازی این پروژه در چهار مرحله به شرح زیر انجام شد:

۱. تحقیق و توسعه

⁸ Protocol Converter Layer

⁹ Hardware Abstraction Layer

¹⁰ Test Execution Engine

¹¹ Framework

فصل اول: کلیات

در مرحله نخست که به تحقیق و توسعه اختصاص داشت، ابتدا مفاهیم اولیه کار با تجهیزات اندازه‌گیری و دستورات کنترل از راه دور آن‌ها مورد تحقیق و یادگیری قرار گرفت. راهکارهای موجود شامل ابزارهای تجاری و کتابخانه‌های متن‌باز مورد بررسی و تحلیل قرار گرفتند، نقاط قوت و ضعف آن‌ها شناسایی شد تا از ایده‌های آن‌ها برای راهکار نهایی استفاده کنیم.

در ادامه، راه‌اندازی اولیه سخت‌افزار از طریق برقراری ارتباط با تجهیزات با استفاده از یک مبدل پورت GPIB خریداری شده انجام شد. در ادامه نیز توسعه نمونه اولیه درایورها صورت گرفت که شامل نوشتن کدهای اولیه برای ارسال دستورات ساده و دریافت پاسخ از دستگاه‌هایی همچون مولتی‌متر و شمارنده بود.

۲. توسعه لایه کنترل سخت‌افزار

در این مرحله، به دلیل ناپایداری اتصال و هزینه بالای تهیه مبدل، یک کنترلر سخت‌افزاری سفارشی^{۱۲} با استفاده از برد رزبری پای و کتابخانه‌های متن‌باز درایور GPIB طراحی و ساخته شد. برای فراهم کردن امکان کنترل از راه دور تجهیزات، یک سرویس نرم‌افزاری در قالب پروکسی شبکه پیاده‌سازی گردید تا کنترلر سخت‌افزاری روی شبکه به اشتراک گذاشته شود. در ادامه، درایورهای سطح بالا تکمیل شدند و کلاس‌های پایتونی جامعی برای هر دستگاه توسعه یافتند که دستورات کنترلی سطح پایین را به توابع معنادار و قابل فهم مانند `measure_voltage` تبدیل می‌کردند.

۳. یکپارچه‌سازی موتور تست و شبیه‌سازی

پس از بررسی چارچوب‌های مختلف در مرحله‌ی تحقیق و توسعه، موتور اجرای آزمون Robot Framework به دلیل توسعه‌پذیری بالا، سادگی در نگارش تست‌ها و مبتنی بودن بر پایتون انتخاب شد. سپس کتابخانه‌های Robot در پایتون توسعه داده شدند تا با ایجاد واسط میان توابع لایه کنترل سخت‌افزار و کلمات کلیدی قابل استفاده در فایل‌های اسکریپت تست، امکان نوشتن تست‌های ساختاریافته فراهم شود. همچنین برای امکان توسعه و تست نرم‌افزار بدون نیاز به دسترسی فیزیکی به تجهیزات از کتابخانه‌های شبیه‌ساز استفاده شد که لایه برقراری ارتباط با تجهیزات را شبیه‌سازی می‌کرد.

۴. توسعه رابط کاربری

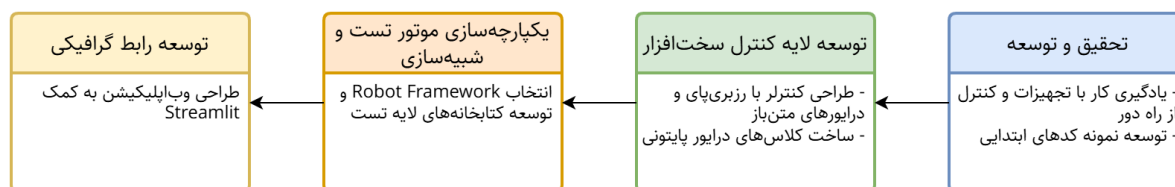
¹² Custom

فصل اول: کلیات

برای مدیریت و نظارت آسان تر بر فرآیند آزمون، یک رابط گرافیکی تحت وب با استفاده از چارچوب ¹³Streamlit طراحی و پیاده سازی شد که وضعیت لحظه ای دستگاه ها را نمایش می دهد و قابلیت پایش را در اختیار کاربر قرار می دهد.

۴-۳-۱ روندنمای کلی پروژه

برای درک بهتر، مراحل اجرای پروژه را می توان در روندنمای زیر خلاصه کرد:



شکل ۱ روندنمای مراحل کلی پیاده سازی

۴-۱ ساختار پروژه

در فصل دوم با مفاهیم و اصطلاحات پایه این پروژه آشنا خواهیم شد و در مورد نحوه کارکرد پروتکل GPIB و مبدل Prologix می خوانیم. پس از آن کتابخانه ها و پروژه های مشابه که مورد بررسی قرار گرفته اند، ذکر خواهند شد. این کتابخانه ها در مرحله تحقیق و توسعه پروژه، نقاط قوت و محدودیت های آن ها شناسایی شده تا از بین آن ها بهترین را انتخاب کنیم یا از ایده های آن ها در پیاده سازی استفاده کنیم. در فصل سوم ساختار معماری با ذکر نمودارهای معماری و پیاده سازی انجام شده مورد بحث قرار خواهد گرفت.

¹³ <https://streamlit.io>

فصل دوم: مفاهیم پایه و کارهای مرتبط

۲-۱ مقدمه

برای درک عمیق معماری و اهداف این پروژه، ابتدا باید با مجموعه‌ای از مفاهیم کلیدی که مفاهیم اصلی این سیستم را تشکیل می‌دهند، آشنا شویم. این فصل به تعریف این مفاهیم بنیادی، از تجهیزات فیزیکی گرفته تا منطق نرم‌افزاری، می‌پردازد تا یک درک پایه‌ای برای مباحث فنی فصل‌های آینده ایجاد گردد.

۲-۱-۱ کلید واژه‌ها

۲-۱-۱-۱ تجهیزات اندازه‌گیری

تجهیزات اندازه‌گیری، ابزارهای سخت‌افزاری هستند که برای سنجش کمیت‌های فیزیکی و الکتریکی به کار می‌روند. دستگاه‌هایی مانند مولتی‌مترهای دیجیتال (برای اندازه‌گیری ولتاژ، جریان و مقاومت)، شمارنده‌ها^{۱۴} (برای اندازه‌گیری فرکانس و دوره تناوب) و منابع تغذیه، نمونه‌هایی از این تجهیزات هستند. به طور معمول، این دستگاه‌ها از طریق پنل جلویی و به صورت دستی توسط کاربر تنظیم و کنترل می‌شوند. در شکل ۲ دستگاه شمارنده (Counter-HP53131A) مشاهده می‌شود که در حال اندازه‌گیری فرکانس است.



شکل ۲ نمای روبه‌روی دستگاه شمارنده در حال نمایش یک اندازه‌گیری فرکانس

¹⁴ Counter

فصل دوم: مفاهیم پایه و کارهای مرتبط



شکل ۳ نمای روبه‌روی دستگاه مولتی‌متر مدل HP3458A که در این پروژه مورد استفاده قرار گرفت

۲-۱-۲ کنترل از راه دور تجهیزات

کنترل از راه دور به قابلیت‌هایی گفته می‌شود که به یک کامپیوتر اجازه می‌دهد تا عملکرد یک دستگاه اندازه‌گیری را بدون دخالت فیزیکی کاربر و از طریق یک رابط ارتباطی (مانند GPIB، CAN، USB یا Ethernet) مدیریت کند. در این حالت، کامپیوتر می‌تواند دستوراتی را برای تنظیم پارامترها ارسال کرده و داده‌های اندازه‌گیری شده را از دستگاه دریافت نماید. این قابلیت، نقش اصلی برای خودکارسازی فرآیندها را ایفا می‌کند. در شکل ۴ نمایی از پشت دستگاه شماره‌دهنده قابل مشاهده است که دارای یک پورت GPIB و RS-232 برای ارتباط با دستگاه برای ارسال دستور و خواندن مقادیر است.



شکل ۴ نمای پشت دستگاه شماره‌دهنده دارای پورت GPIB

۲-۱-۱-۳ مبدل پروتکل

بسیاری از تجهیزات اندازه‌گیری قدیمی‌تر، تنها از پروتکل ارتباطی GPIB پشتیبانی می‌کنند. از آنجایی که کامپیوترها فاقد این پورت هستند، مبدل GPIB به عنوان یک پل سخت‌افزاری عمل می‌کند. این دستگاه، سیگنال‌های پروتکل GPIB را به یک پروتکل رایج‌تر مانند Ethernet (شبکه) یا USB ترجمه می‌کند. این فرآیند به کامپیوترهای امروزی اجازه می‌دهد تا با تجهیزات قدیمی‌تر به صورت یکپارچه و از راه دور ارتباط برقرار کنند.

۲-۱-۱-۴ سامانه آزمون خودکار

یک سامانه آزمون خودکار، مجموعه‌ای یکپارچه از نرم‌افزار و سخت‌افزار است که برای اجرای آزمون‌ها بر روی یک محصول (DUT)^{۱۵} بدون نیاز به دخالت انسان طراحی شده است. این سامانه با استفاده از قابلیت کنترل از راه دور، یک یا چند دستگاه اندازه‌گیری را برای اندازه‌گیری پیکربندی می‌کند، نتایج را جمع‌آوری کرده، آن‌ها را با مقادیر مورد انتظار مقایسه نموده و در نهایت گزارش تولید می‌کند. هدف اصلی چنین سامانه‌ای، افزایش سرعت، دقت و تکرارپذیری آزمون‌ها است.

۲-۱-۱-۵ سناریوی آزمون^{۱۶}

سناریوی تست، مجموعه‌ای از دستورالعمل‌ها و گام‌های از پیش تعریف‌شده است که یک سامانه آزمون خودکار باید به ترتیب اجرا کند تا یک آزمون خاص کامل شود. این سناریو به صورت یک فایل یا اسکریپت نوشته می‌شود و جزئیاتی مانند موارد زیر را مشخص می‌کند:

- کدام دستگاه‌ها باید تنظیم شوند.
 - پارامترهای هر دستگاه چگونه باید پیکربندی شوند (مثلاً برای دستگاه منبع تغذیه ولتاژ روی ۱۰ ولت تنظیم شود).
 - چه مقادیری باید اندازه‌گیری شوند.
 - معیارهای موفقیت^{۱۷} یا شکست^{۱۸} آزمون چیست (مثلاً ولتاژ اندازه‌گیری شده باید بین ۴.۹ تا ۵.۱ ولت باشد).
- و دوباره این چرخه از بالا تا پایین برای یک اندازه‌گیری دیگر طی می‌شود.

¹⁵ Device Under Test

¹⁶ Test Scenario

¹⁷ Pass

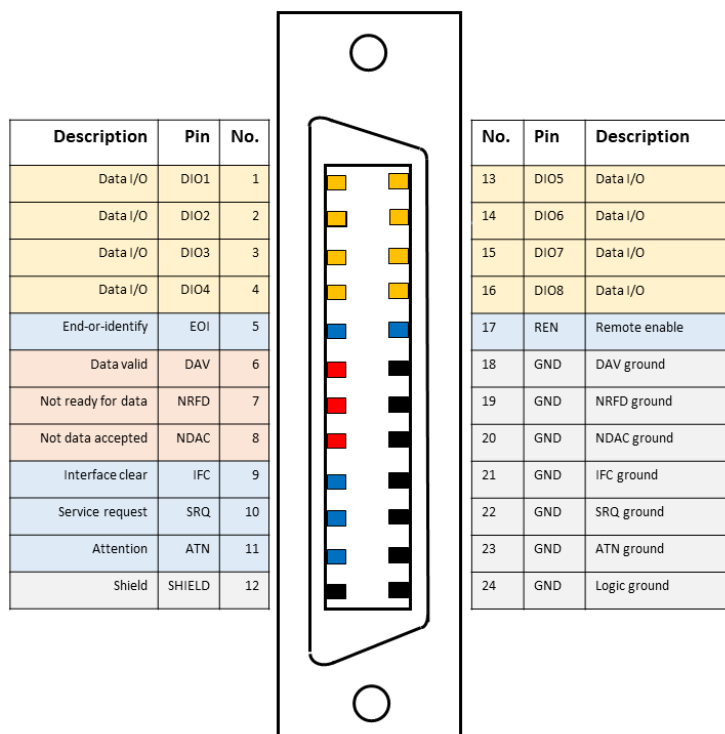
¹⁸ Fail

۲-۱-۲ پروتکل GPIB

پروتکل GPIB¹⁹، که با استاندارد IEEE-488 نیز شناخته می‌شود، یک رابط استاندارد برای اتصال تجهیزات اندازه‌گیری و کنترلی است. این رابط از یک کانکتور ۲۴ پین بهره می‌برد که شامل ۸ خط داده موازی، ۳ خط برای Handshake و ۵ خط فرمان برای مدیریت ارتباط است.

ویژگی کلیدی GPIB، مکانیزم Handshake آن است که تضمین می‌کند داده‌ها تنها زمانی ارسال می‌شوند که گیرنده آماده دریافت باشد و داده ارسالی توسط گیرنده دریافت شده باشد. این مکانیزم، یک ارتباط پایدار و قابل اطمینان را بین دستگاه‌های مختلف روی باس فراهم می‌کند و از گم شدن داده‌ها در حین انتقال جلوگیری می‌کند.

در شکل پایین نمای پین‌های پورت GPIB قابل مشاهده است:



شکل ۵ نمای پین‌های کانکتور پورت GPIB

¹⁹ General Purpose Interface Bus

فصل دوم: مفاهیم پایه و کارهای مرتبط

در پروتکل GPIB، دستگاه‌های متصل به باس می‌توانند یکی از سه نقش اصلی زیر را ایفا کنند:

- **کنترل‌کننده^{۲۰}:** این دستگاه، مدیر و هماهنگ‌کننده اصلی باس است. وظیفه آن تعیین دستگاهی است که باید داده ارسال کند (Talker) و دستگاه‌هایی که باید داده دریافت کنند (Listeners). مثلاً در این پروژه مبدل استفاده شده برای تبدیل پروتکل Ethernet به GPIB این نقش را بر عهده دارد.
- **ارسال‌کننده^{۲۱}:** دستگاهی که در یک لحظه مشخص، مجوز ارسال داده بر روی باس را از کنترل‌کننده دریافت کرده است. به عنوان مثال، یک مولتی‌متر که مقدار ولتاژ اندازه‌گیری شده را ارسال می‌کند، در نقش Talker عمل می‌کند.
- **دریافت‌کننده^{۲۲}:** یک یا چند دستگاه که برای دریافت داده از Talker توسط کنترل‌کننده آدرس‌دهی شده‌اند. برای نمونه، یک منبع تغذیه که تنظیمات جدید ولتاژ را از کامپیوتر دریافت می‌کند، یک Listener است.

سیگنال‌های فرمان جدول ۱، وظیفه‌ی همگام‌سازی و تضمین انتقال صحیح هر بایت از داده را بر عهده دارند.

جدول ۱ سیگنال‌های Handshake در GPIB

سیگنال	نام کامل	جهت سیگنال	عملکرد
DAV	Data valid	ارسال‌کننده به دریافت‌کننده	Talker این خط را فعال می‌کند تا اعلام کند یک بایت داده آماده و معتبر است.
NRFD	Not ready for data	دریافت‌کننده به ارسال‌کننده	Listener این خط را تا زمانی که برای دریافت بایت آماده شود، فعال نگه می‌دارد.
NDAC	Not data accepted	دریافت‌کننده به ارسال‌کننده	Listener این خط را تا زمانی که بایت داده را با موفقیت بپذیرد، فعال نگه می‌دارد.

سیگنال‌های فرمان جدول ۲ برای دستورات سطح بالا، کنترل باس و آدرس‌دهی دستگاه‌ها استفاده می‌شود.

²⁰ Controller

²¹ Talker

²² Listener

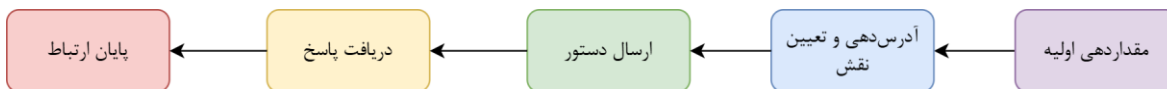
فصل دوم: مفاهیم پایه و کارهای مرتبط

جدول ۲ سیگنال‌های کنترل و مدیریت باس GPIB

سیگنال	نام کامل	جهت سیگنال	عملکرد
ATN	Attention	کنترلر به دستگاه‌ها	باس را در حالت فرمان قرار می‌دهد. وقتی فعال باشد، کنترلر در حال ارسال فرمان یا آدرس است و اگر غیرفعال باشد در حال ارسال داده.
IFC	Interface Clear	کنترلر به دستگاه‌ها	کل باس GPIB را ریست می‌کند. برای مقداردهی اولیه هر چیزی استفاده می‌شود.
SRQ	Service Request	دستگاه به کنترلر	دستگاه سیگنال وقفه ^{۲۳} می‌دهد که به رسیدگی نیاز دارد (مثلاً اندازه‌گیری کامل شده یا خطایی رخ داده است).
REN	Remote Enable	کنترلر به دستگاه‌ها	دستگاه را به حالت کنترل از راه دور می‌برد.
EOI	End Or Identify	ارسال کننده به دریافت کننده	برای مشخص کردن آخرین بایت بسته پیام استفاده می‌شود.

۲-۱-۲-۱ فرآیند یک ارتباط نمونه در باس GPIB

در ادامه، یک نمونه از مراحل ارتباط بین یک کنترل کننده و یک دستگاه (مانند مولتی متر) بیان می‌شود:



شکل ۶ فرآیند ارسال یک دستور به دستگاه GPIB

۱. مقداردهی اولیه:

- کنترل کننده خط IFC (Interface Clear) را فعال می‌کند تا کل باس را ریست کرده و تمام دستگاه‌ها را به یک حالت اولیه مشخص بازگرداند.

۲. آدرس دهی و تعیین نقش:

- کنترل کننده خط ATN (Attention) را فعال می‌کند تا به همه دستگاه‌ها اعلام کند که اطلاعات روی خطوط داده، فرمان یا آدرس است، نه داده‌ی پیام.

²³ Interrupt

فصل دوم: مفاهیم پایه و کارهای مرتبط

- کنترل کننده آدرس دستگاه مورد نظر (مثلا مولتی متر) را روی باس ارسال کرده و به آن نقش دریافت کننده را اختصاص می دهد تا آماده دریافت فرمان شود.
- در این حالت، خود کنترل کننده نقش ارسال کننده را بر عهده می گیرد.

۳. ارسال فرمان:

- کنترل کننده فرمان مورد نظر (مثلا MEAS:VOLT? برای اندازه گیری ولتاژ) را از طریق ۸ خط داده ارسال می کند.
- انتقال هر بایت از فرمان با استفاده از سیگنال های handshake (DAV, NRFD, NDAC) انجام می شود تا از تحویل صحیح و پایدار آن اطمینان حاصل شود.

۴. دریافت پاسخ:

- پس از ارسال کامل فرمان، کنترل کننده نقش ها را معکوس می کند یعنی به دستگاه مولتی متر فرمان می دهد که Talker شود و خود نقش Listener می شود.
- دستگاه پاسخ را (مثلا مقدار ۲۵.۶۱۲) بر روی باس ارسال می کند.
- با ارسال آخرین بایت، دستگاه خط EOI (End or Identify) را فعال می کند تا پایان انتقال پیام را به کنترل کننده اطلاع دهد.

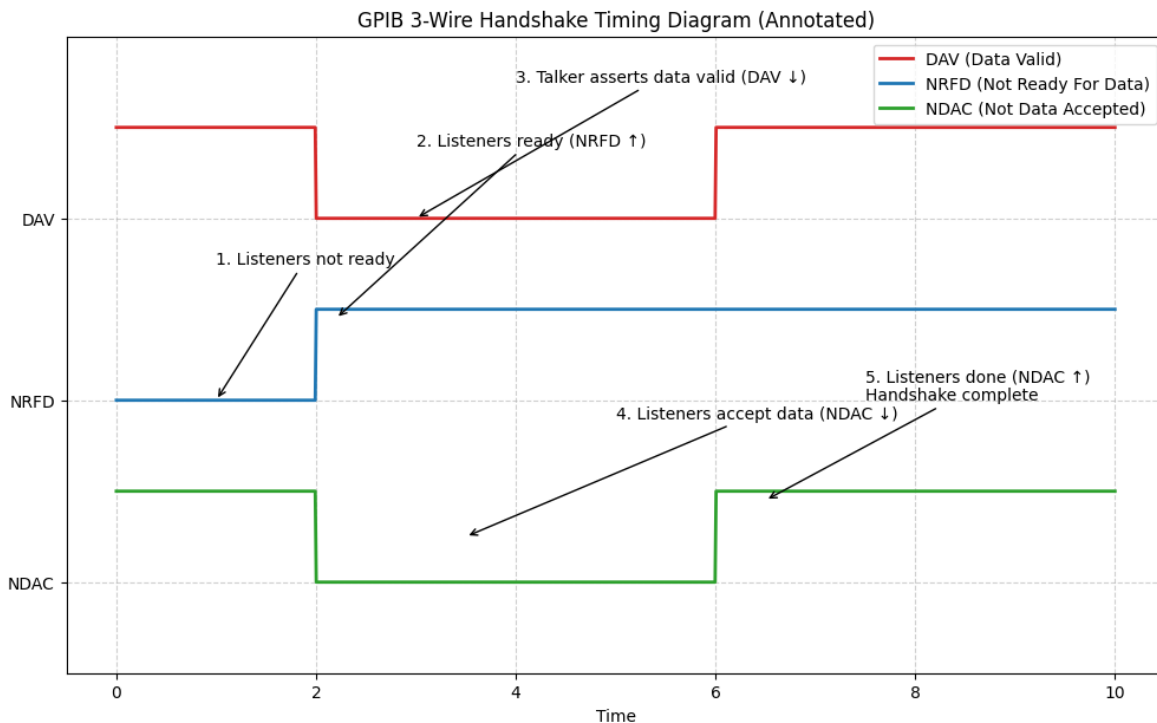
۵. پایان ارتباط:

- در نهایت، کنترل کننده با ارسال فرمان های UNL (Unlisten) و UNT (Untalk)، دستگاه را از حالت آدرس دهی خارج کرده و باس را آزاد می کند.
- در این نقطه، ارتباط به پایان می رسد و کنترل کننده می تواند ارتباط جدیدی را با دستگاه دیگری آغاز کند.

فصل دوم: مفاهیم پایه و کارهای مرتبط

۲-۱-۲-۲ فرآیند Handshake در باس GPIB

در شکل پایین روند تغییرات سیگنال‌های Handshake بر حسب زمان برای دریافت یک بایت داده نمایش داده شده است که صحت ارسال داده را تضمین می‌کند.



شکل ۷ نمودار تغییرات سیگنال‌های Handshake باس GPIB بر حسب زمان

۱. آماده‌سازی اولیه

- ارسال‌کننده یک بایت داده را روی ۸ خط داده قرار می‌دهد.
- در این حالت، دریافت‌کننده خط NRFD (عدم آمادگی برای داده) را فعال نگه داشته است تا نشان دهد هنوز آماده دریافت نیست.

۲. اعلام آمادگی توسط دریافت‌کننده

- دریافت‌کننده به محض آمادگی برای دریافت داده، خط NRFD را رها می‌کند.

۳. معتبرسازی داده توسط ارسال‌کننده

- ارسال‌کننده با تشخیص غیرفعال شدن خط NRFD، متوجه می‌شود که دریافت‌کننده آماده است.
- سپس، خط DAV (داده معتبر) را فعال می‌کند تا به اعلام کند که داده‌ی روی باس، پایدار و معتبر است.

۴. دریافت و پذیرش داده

- دریافت‌کننده با تشخیص فعال شدن خط DAV، داده را از روی خطوط می‌خواند.
- پس از خواندن موفقیت‌آمیز داده، دریافت‌کننده خط NDAC (عدم پذیرش داده) را رها می‌کند.

۵. پایان چرخه انتقال

- ارسال‌کننده با تشخیص غیرفعال شدن خط NDAC، متوجه می‌شود که دریافت‌کننده داده را پذیرفته است.
- در نهایت، ارسال‌کننده خط DAV را غیرفعال می‌کند و می‌تواند چرخه‌ی جدیدی را برای ارسال بایت بعدی آغاز کند.

۳-۱-۲ مبدل GPIB-Ethernet شرکت Prologix

مبدل GPIB-Ethernet شرکت Prologix یک دستگاه سخت‌افزاری است که به عنوان پلی بین شبکه TCP/IP و باس قدیمی GPIB عمل می‌کند. این مبدل به تجهیزات اندازه‌گیری که فقط دارای پورت GPIB هستند، اجازه می‌دهد تا از طریق یک شبکه استاندارد Ethernet کنترل شوند. این ویژگی، امکان کنترل از راه دور و توزیع‌شده تجهیزات را فراهم می‌کند.

۳-۱-۲-۱ نحوه عملکرد



شکل ۸ نمای جلو و عقب مبدل Prologix

این مبدل به شبکه متصل شده و یک آدرس IP دریافت می‌کند. سپس، روی یک پورت مشخص (معمولاً ۱۲۳۴) منتظر اتصالات TCP از سمت کامپیوتر کنترل‌کننده می‌ماند. نرم‌افزار کنترلی (مانند یک کد پایتون) یک ارتباط سوکت با این آدرس IP و پورت برقرار می‌کند. پس از برقراری ارتباط، نرم‌افزار می‌تواند دو نوع دستور برای مبدل ارسال کند:

- **دستورات ویژه مبدل:** این دستورات که با ++ شروع می‌شوند، برای خود مبدل ارسال شده و وظیفه تنظیم و مدیریت آن را بر عهده دارند.

فصل دوم: مفاهیم پایه و کارهای مرتبط

- **دستورات استاندارد GPIB:** این دستورات، فرمان‌های واقعی تجهیزات هستند (مانند ?IDN*) که مبدل آن‌ها

را مستقیماً به دستگاهی که از قبل آدرس‌دهی شده، ارسال می‌کند.

۲-۱-۳-۲ ساختار دستورات ویژه مبدل

در جدول ۳، برخی از مهم‌ترین دستورات ویژه Prologix تشریح شده‌اند:

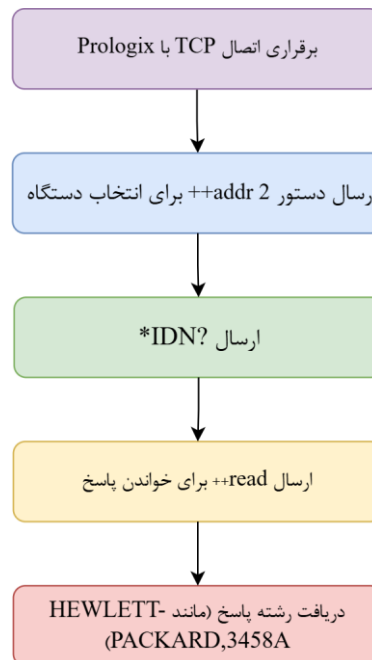
جدول ۳ برخی از دستورات مبدل Prologix

دستور	عملکرد
++addr	آدرس دستگاه هدف روی بوس GPIB را تنظیم می‌کند. تمام دستورات بعدی به این دستگاه ارسال خواهند شد. مثال: ++addr 5
++read	به صورت دستی مبدل را وادار به خواندن پاسخ از دستگاه می‌کند. معمولاً زمانی استفاده می‌شود که ++auto غیرفعال باشد.
++ver	نسخه Firmware مبدل را نمایش می‌دهد.
++srq	وضعیت خط SRQ را بررسی کرده و مقدار 1 (فعال) یا 0 (غیرفعال) را برمی‌گرداند.
++auto	حالت خواندن خودکار را کنترل می‌کند. اگر فعال باشد، مبدل پس از ارسال یک فرمان، به طور خودکار منتظر پاسخ دستگاه می‌ماند.

فصل دوم: مفاهیم پایه و کارهای مرتبط

۲-۱-۳-۳ نمونه فرآیند ارتباط

در نمودار شکل پایین نمونه‌ای از فرآیند ارسال دستور و دریافت پاسخ مشخص شده است.



شکل ۹ ترتیب فرآیند ارسال یک دستور و دریافت پاسخ به کمک مبدل Prologix

برای خواندن شناسه یک دستگاه با آدرس ۲ روی باس، مراحل زیر طی می‌شود:

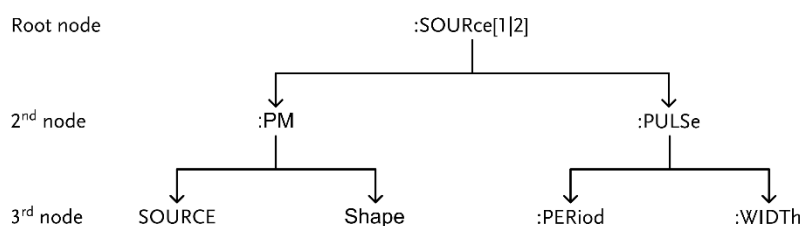
۱. برقراری اتصال TCP با آدرس IP مبدل Prologix.
۲. ارسال دستور ++addr 2 برای انتخاب دستگاه مورد نظر.
۳. ارسال دستور *IDN? که مبدل آن را به دستگاه با آدرس ۲ می‌فرستد.
۴. ارسال دستور ++read تا مبدل پاسخ دستگاه را بخواند.
۵. دریافت رشته شناسه دستگاه (مثلاً HEWLETT-PACKARD,3458A) از طریق سوکت.

۴-۱-۲ دستورات استاندارد برای تجهیزات قابل برنامه‌ریزی (SCPI)

SCPI²⁴ یک زبان استاندارد است که برای کنترل تجهیزات اندازه‌گیری و آزمون خودکار طراحی شده است. هدف اصلی SCPI، ایجاد یک قاعده مشترک و سازگار برای کنترل دستگاه‌های مختلف از تولیدکنندگان متفاوت است. این استاندارد بر پایه پیام‌های متنی ASCII بنا شده و معمولاً از طریق رابط‌هایی مانند GPIB، USB، Ethernet یا Serial ارسال می‌شود.

۴-۱-۲-۱ ساختار درختی دستورات

یکی از ویژگی‌های SCPI، ساختار سلسله‌مراتبی یا درختی آن است. دستورات در یک ساختار منطقی سازماندهی شده‌اند که عملکرد دستگاه را مدل‌سازی می‌کند. همانطور که در شکل پایین مشخص است، هر کلمه کلیدی در یک دستور، یک گره²⁵ در این درخت محسوب می‌شود و زیرگره‌های مختلف با استفاده از کاراکتر دونقطه (:) از یکدیگر جدا می‌شوند.



شکل ۱۰ ساختار درختی دستورات SCPI

مثال، دستور 5ms :SOURCE:PULSe:PERiod را در نظر بگیرید:

- :SOURCE: گره ریشه است که به زیرسیستم منبع سیگنال اشاره دارد.
- :PULSe: زیرمجموعه‌ای از :SOURCE است و مربوط به تنظیمات پالس می‌شود.
- :PERiod: گره نهایی است که دوره تناوب پالس را مشخص می‌کند.

۴-۱-۲-۲ انواع دستورات

دستورات SCPI به طور کلی به دو دسته اصلی تقسیم می‌شوند:

- **دستورات تنظیم‌کننده²⁶:** این دستورات برای تنظیم یک پارامتر یا اجرای یک عملیات در دستگاه استفاده می‌شوند و معمولاً مقداری را برنمی‌گردانند.

²⁴ Standard Commands for Programmable Instruments

²⁵ Node

²⁶ Setter Commands

فصل دوم: مفاهیم پایه و کارهای مرتبط

مثال: `OUTPut:STATe ON` (خروجی دستگاه را فعال می‌کند).

- **دستورات کوئری^{۲۷}:** این دستورات برای خواندن وضعیت یا مقدار یک پارامتر از دستگاه به کار می‌روند. مشخصه اصلی آن‌ها وجود یک علامت سؤال (?) در انتهای دستور است. این دستورات همیشه یک پاسخ از دستگاه را به همراه دارند.

مثال: `MEASure:VOLTage:DC?` (از دستگاه می‌خواهد ولتاژ DC را اندازه‌گیری کرده و نتیجه را برگرداند).

- گاهی یک دستور می‌تواند هر دو حالت را داشته باشد. برای مثال `SOURce:FREQuency 10` فرکانس خروجی سیگنال را روی ۱۰ هرتز تنظیم می‌کند، در حالی که `SOURce:FREQuency?` مقدار تنظیم شده فعلی را برمی‌گرداند.

۲-۱-۴-۳ پارامترها و سینتکس

- **پارامترها:** بسیاری از دستورات برای تکمیل عملکرد خود به یک یا چند پارامتر نیاز دارند که پس از یک فاصله بعد از دستور قرار می‌گیرند. این پارامترها می‌توانند اجباری یا اختیاری باشند. پارامترهای اختیاری معمولاً دارای مقدار پیش‌فرض هستند که در صورت عدم ارسال، دستگاه آن را در نظر می‌گیرد.

مثال: `SOURce:FREQuency <frequency>` که `<frequency>` یک پارامتر اجباری است.

- **فرم کوتاه و بلند:** برای افزایش خوانایی و همچنین کاهش حجم داده‌های ارسالی، هر کلمه کلیدی در SCPI یک فرم بلند و یک فرم کوتاه دارد. فرم کوتاه معمولاً با حروف بزرگ مشخص می‌شود. دستگاه هر دو فرم را می‌پذیرد.
`FREQuency` (فرم بلند) و `FREQ` (فرم کوتاه).

۲-۱-۴-۴ دستورات مشترک

استاندارد SCPI مجموعه‌ای از دستورات را تعریف می‌کند که با علامت ستاره (*) شروع می‌شوند و باید در تمام دستگاه‌های سازگار با SCPI عملکرد یکسانی داشته باشند. این دستورات به هیچ زیرسیستم خاصی تعلق ندارند و در ریشه درخت دستورات قرار دارند. برخی از این دستورات در جدول ۴ ذکر شده‌اند:

²⁷ Query Commands

فصل دوم: مفاهیم پایه و کارهای مرتبط

جدول ۴ دستورات مشترک و استاندارد SCPI

دستور	نام کامل	عملکرد
*IDN?	Identification Query	شناسه دستگاه (شامل نام سازنده، مدل، شماره سریال و نسخه Firmware) را برمی گرداند.
*RST	Reset	دستگاه را به تنظیمات پیش فرض کارخانه بر می گرداند.
*CLS	Clear Status	تمام رجیسترهای وضعیت و صف خطاها را پاک می کند.
*OPC	Operation Complete	پس از اتمام تمام دستورات قبلی، یک بیت مشخص را در رجیستر وضعیت تنظیم می کند.

۵-۱-۲ مطالعات انجام شده روی تجهیزات

بخشی از زمان پروژه صرف مطالعه و یادگیری کار با تجهیزات اندازه گیری شد. برای ساخت یک سامانه کنترل از راه دور نیازمند آشنایی با مفاهیم و کاربردهای هر یک از دستگاه ها بودیم. از آنجایی که این دستگاه ها از استانداردهای دستوری مشخصی پیروی می کنند، آشنایی با یک دستگاه، مطالعه و یادگیری دستگاه های بعدی را سریع تر می کند. در این پروژه ۴ دستگاه مورد مطالعه و پیاده سازی قرار گرفت که در ادامه ذکر می شوند.

۱-۵-۲ دستگاه مولتی متر

مولتی متر دیجیتال HP-3458A به عنوان یک استاندارد مرجع در صنعت شناخته می شود و با دقت ۸.۵ رقم، دقت بسیار بالایی برای اندازه گیری های ولتاژ، جریان، مقاومت و فرکانس فراهم می کند. این دستگاه پیچیدگی و تنوع تنظیمات بسیار زیادی دارد که نیازمند مطالعه مستندات آن است. در این بخش، امکانات و کاربردهای خاص این دستگاه در اندازه گیری که مورد مطالعه قرار گرفته است، شرح داده می شود.

۱. اندازه گیری دسته ای

فرآیند انجام یک اندازه گیری در این دستگاه از یک توالی رویداد مشخص پیروی می کند که شامل آماده سازی تریگر (Trigger Arm)، وقوع رویداد تریگر (Trigger Event) و انجام اندازه گیری (Measurement Event) است. این قابلیت وجود دارد که به کمک دستور TIMER می توان فاصله زمانی وقوع این رویدادها را تنظیم کرد و به جای دریافت یک مقدار اندازه گیری، چندین اندازه گیری با فاصله زمانی مشخص انجام داد و در خروجی دریافت کرد.

۲. دقت اندازه‌گیری

پارامتر زمان انتگرال‌گیری^{۲۸} مشخص می‌کند که مبدل آنالوگ به دیجیتال دستگاه چه مدتی از سیگنال ورودی نمونه‌برداری کند. زمان طولانی‌تر به معنای حذف بهتر نویز (به‌ویژه نویز شبکه برق) و دقت بالاتر است. این پارامتر از طریق دو دستور قابل تنظیم است:

- NPLC (Number of Power Line Cycles): زمان انتگرال‌گیری را بر حسب تعداد سیکل‌های شبکه برق (مثلاً ۵۰ یا ۶۰ هرتز) تنظیم می‌کند. مقدار کمتر، سرعت بیشتر و دقت کمتری را به همراه دارد.
- APER (Aperture): مستقیماً زمان انتگرال‌گیری را بر حسب ثانیه تنظیم می‌کند.
- رزولوشن (Resolution): این پارامتر نیز مستقیماً با زمان انتگرال‌گیری در ارتباط است. انتخاب رزولوشن بالاتر، دستگاه را مجبور به استفاده از زمان انتگرال‌گیری طولانی‌تر می‌کند.

۳. افزایش دقت

قابلیت Autozero برای حذف خطاهای آفست^{۲۹} داخلی مولتی‌متر طراحی شده است. با فعال بودن آن، دستگاه قبل از هر اندازه‌گیری، ورودی خود را قطع کرده و یک اندازه‌گیری از صفر داخلی انجام می‌دهد و سپس این مقدار خطا را از نتیجه نهایی کم می‌کند. این کار دقت را به شکل چشمگیری افزایش می‌دهد ولی زمان هر اندازه‌گیری را تقریباً دو برابر می‌کند. دستگاه به طور پیش‌فرض می‌تواند به صورت خودکار بهترین بازه اندازه‌گیری را انتخاب کند. در حالی که این قابلیت برای کاربردهای عمومی مفید است، فرآیند تصمیم‌گیری آن باعث افزایش زمان اندازه‌گیری می‌شود. غیرفعال کردن قابلیت Autorange و تنظیم یک بازه ثابت، سرعت اندازه‌گیری را افزایش می‌دهد.

۴. مدیریت وضعیت

مولتی‌متر دارای رجیسترهای وضعیت ۸ بیتی است که اطلاعاتی مانند آماده بودن داده، وقوع خطا و... را در خود نگه می‌دارند. بیت هفتم این رجیستر، آماده بودن داده در بافر خروجی را نشان می‌دهد. با برنامه‌ریزی مناسب، می‌توان دستگاه را طوری تنظیم کرد که پس از آماده شدن داده، از طریق خط SRQ به کنترلر وقفه ارسال کند.

²⁸ Integration Time

²⁹ Offset

فصل دوم: مفاهیم پایه و کارهای مرتبط

با فعال‌سازی بافر ورودی، کنترلر می‌تواند مجموعه‌ای از دستورات را به سرعت برای دستگاه ارسال کند، حتی اگر دستگاه در حال انجام یک اندازه‌گیری طولانی باشد. دستگاه دستورات را در بافر خود ذخیره کرده و به ترتیب اجرا می‌کند. این قابلیت از اشغال شدن باس GPIB جلوگیری کرده و کارایی سیستم را بالا می‌برد.

۲-۵-۱-۲ دستگاه شمارنده

شمارنده^{۳۰} مدل HP-53131A یکی از تجهیزات است که برای اندازه‌گیری پارامترهای مرتبط با زمان و فرکانس سیگنال‌های الکتریکی به کار می‌رود. در طول این پروژه، قابلیت‌های این دستگاه مورد مطالعه و استفاده قرار گرفت که در ادامه شرح داده می‌شوند.

۱. قابلیت‌های اصلی اندازه‌گیری

این دستگاه مجموعه‌ای از اندازه‌گیری‌های استاندارد را فراهم می‌کند:

- اندازه‌گیری فرکانس^{۳۱}: اصلی‌ترین کاربرد دستگاه، اندازه‌گیری فرکانس سیگنال ورودی است.
- اندازه‌گیری دوره تناوب^{۳۲}: دستگاه قادر است دوره تناوب یک سیگنال، یعنی زمان یک سیکل کامل آن را اندازه‌گیری کند.
- اندازه‌گیری بازه زمانی^{۳۳}: این قابلیت امکان اندازه‌گیری دقیق فاصله زمانی بین دو رویداد^{۳۴} را فراهم می‌کند. این رویدادها می‌توانند روی یک کانال ورودی (مثلاً بین لبه بالارونده و پایین‌رونده یک پالس) یا بین دو کانال مختلف تعریف شوند.
- قابلیت شمارنده (Totalize): علاوه بر اندازه‌گیری‌های مبتنی بر زمان، این دستگاه می‌تواند به عنوان یک شمارنده رویداد عمل کند. در حالت Totalize، دستگاه تعداد پالس‌ها یا رویدادهایی که از یک آستانه ولتاژ مشخص عبور می‌کنند را در یک بازه زمانی معین می‌شمارد.

۲. قابلیت‌های پیکربندی اندازه‌گیری

³⁰ Counter

³¹ Frequency Measurement

³² Period Measurement

³³ Time Interval

³⁴ Event

فصل دوم: مفاهیم پایه و کارهای مرتبط

این دستگاه تنظیمات متنوعی برای افزایش دقت و پایداری اندازه‌گیری‌ها فراهم می‌کند:

- میانگین‌گیری^{۳۵}: دستگاه این امکان را فراهم می‌کند که از چندین اندازه‌گیری متوالی میانگین بگیرد. قابلیت‌های Period Average و Time Interval Average با حذف نوسانات تصادفی، یک نتیجه پایدارتر از دوره تناوب یا بازه زمانی ارائه می‌دهند.
- پیکربندی رویداد تریگر (Trigger Event): برای هر کانال ورودی می‌توان مشخص کرد که رویداد مورد نظر، لبه بالارونده^{۳۶} یا لبه پایین‌رونده^{۳۷} سیگنال باشد. این قابلیت برای اندازه‌گیری پارامترهایی مانند عرض پالس^{۳۸} مهم است.
- کوپلینگ ورودی (Input Coupling): کاربر می‌تواند نوع کوپلینگ ورودی را بین AC و DC انتخاب کند. کوپلینگ DC کل سیگنال (شامل مولفه DC) را عبور می‌دهد، در حالی که کوپلینگ AC تنها مولفه متناوب سیگنال را برای اندازه‌گیری در نظر می‌گیرد و اثر آفست DC را حذف می‌کند.
- همگام‌سازی^{۳۹} با استفاده از وقفه‌ها
یکی از چالش‌ها، اطلاع از زمان آماده شدن نتیجه اندازه‌گیری است. به جای استفاده از تاخیرهای ثابت و غیربهبوده Polling، از مکانیزم وقفه داخلی دستگاه استفاده شد. دستگاه HP-53131A قادر است پس از اتمام یک اندازه‌گیری، از طریق خط SRQ (Service Request) در باس GPIB به کنترل‌کننده اطلاع دهد. با تنظیم رجیسترهای وضعیت^{۴۰} دستگاه، می‌توان آن را طوری تنظیم کرد که تنها در صورت آماده شدن داده جدید، این سیگنال وقفه را تولید کند.

³⁵ Averaging

³⁶ Rising Edge

³⁷ Falling Edge

³⁸ Pulse Width

³⁹ Synchronization

⁴⁰ Status Registers

۳-۵-۲ دستگاه تولیدکننده سیگنال

دستگاه AFG-2225 یک تولیدکننده سیگنال^{۴۱} دو کاناله است که برای تولید طیف گسترده‌ای از شکل‌موج‌های استاندارد مانند سینوسی، مربعی و پالسی به کار می‌رود. همانطور که در شکل ۱۲ مشاهده می‌شود این دستگاه برای کنترل از راه دور، از یک رابط USB بهره می‌برد. این پورت USB به نرم‌افزارهای کنترلی اجازه می‌دهد تا با دستگاه ارتباط برقرار کرده و فرمان‌های SCPI را برای تنظیم فرکانس، دامنه، شکل‌موج و سایر پارامترها ارسال نمایند. در شکل ۱۱ نمای روبه‌روی دستگاه قابل مشاهده است.

در این پروژه از این دستگاه برای تولید سیگنال ورودی دستگاه شمارنده استفاده شد تا دستگاه شمارنده مورد بررسی قرار بگیرد. هر چند که این دستگاه جزو نیازمندی‌های پروژه نبود ولی برای واقعی‌تر شدن آزمون‌ها، با سامانه کنترل از راه دور مانند دستگاه‌های قبلی یکپارچه شد تا یک سناریو واقعی از تنظیم خودکار یک دستگاه تولیدکننده سیگنال شروع شود و این سیگنال توسط دیگر دستگاه‌ها اندازه‌گیری شود.



شکل ۱۱ نمای روبه‌روی دستگاه تولیدکننده سیگنال — Function generator AFG2225

⁴¹ Function Generator

فصل دوم: مفاهیم پایه و کارهای مرتبط



شکل ۱۲ نمای پشت دستگاه تولیدکننده سیگنال که از طریق پورت USB قابل کنترل از راه دور است

۲-۱-۵-۴ بررسی قابلیت‌های Power meter

دستگاه HP-E4419B یک اندازه‌گیر توان^{۴۲} است که برای اندازه‌گیری توان سیگنال‌های فرکانس بالا استفاده می‌شود. این دستگاه به تنهایی عمل نمی‌کند و برای انجام اندازه‌گیری نیازمند اتصال به یک سنسور خارجی است. این دستگاه مانند دستگاه‌های مولتی‌متر و شمارنده از پروتکل GPIB استفاده می‌کند. این دستگاه برای کاهش خطا اندازه‌گیری سنسور فرآیندی به نام zero-adjust را می‌تواند اجرا کند. در این فرآیند دستگاه با قطع اتصال سنسور، یک اندازه‌گیری انجام می‌دهد که مقدار حاصل مقدار خطای اندازه‌گیری خواهد بود. از آن پس بعد از هر اندازه‌گیری این مقدار خطا را از مقدار دریافتی از سنسور کم می‌کند.



شکل ۱۳ نمای روبه‌روی دستگاه اندازه‌گیر توان – Power meter E4419B

⁴² Power Meter

۲-۲ تحلیل نقاط قوت و ضعف منابع غیر پژوهشی مشابه

در این فصل به تحلیل ابزارها و کتابخانه‌ها می‌پردازیم. در این پروژه از چند دسته کتابخانه استفاده کرده‌ایم:

- ابزارهای کنترل تجهیزات
- ابزارهای اجرای آزمون و خودکارسازی اندازه‌گیری
- ابزارهای رابط گرافیکی
- ابزارهای ارتباطی
- ابزارهای شبیه‌سازی

۲-۲-۱ ابزارهای کنترل تجهیزات

تجهیزات اندازه‌گیری آزمایشگاهی دستورات خود را به صورت رشته‌ای از کاراکترها دریافت می‌کنند. گاهی برای تنظیم یک دستگاه باید چندین دستور مختلف را ارسال کرد و سپس مقدار اندازه‌گیری شده را خواند. به همین خاطر وجود یک فرآیند تکرارپذیر و سطح بالا بسیار کاربردی است. از این رو کتابخانه‌های مختلفی با عنوان ایجاد یک لایه انتزاعی برای تجهیزات توسعه یافته‌اند. هدف این کتابخانه‌ها، نگاشت هر دستگاه به یک شی در زبان شی‌گرایی مانند پایتون است که هر شی رفتاری مثل اندازه‌گیری ولتاژ، فرکانس و... دارد. از آن پس توسعه‌دهنده‌ی کد به جای ارسال دستورات سطح پایین رشته‌ای به دستگاه‌ها، یک متد معنادار مانند `measure_voltage` را فراخوانی می‌کند و عدد اندازه‌گیری شده برگردانده می‌شود.

۲-۲-۱-۱ کتابخانه Pymeasure

⁴³Pymeasure یک کتابخانه‌ی متن‌باز برای تعامل با ابزارهای اندازه‌گیری و نمایش داده‌ها است که امکاناتی برای کنترل تجهیزات و رسم نمودار گرافیکی (بر پایه Qt) فراهم می‌کند. این کتابخانه جامع‌ترین و پیشرفته‌ترین کتابخانه موجود است که در حال حاضر از همین کتابخانه در پروژه استفاده شده است.

⁴³ <https://github.com/pymasure/pymasure>

نقاط قوت:

- پوشش نسبتاً گسترده‌ای از دستگاه‌ها و تجهیزات اندازه‌گیری دارد.
- رابط گرافیکی برای نمایش نموداری مقادیر اندازه‌گیری شده دارد.
- کامیت‌های اخیر آن نشان می‌دهد پروژه‌ی بسیار فعالی است.
- مستندسازی رسمی و مثال‌های کاربردی مناسبی دارد.
- استفاده از کتابخانه ارتباطی PyVISA که پروتکل ارسال و دریافت داده‌ها را از لایه‌های بالاتر مخفی می‌کند.
- برای اطمینان از صحت عملکرد متدها (مانند یک متد اندازه‌گیری ولتاژ) امکان نوشتن تست برای هر دستگاه وجود دارد.

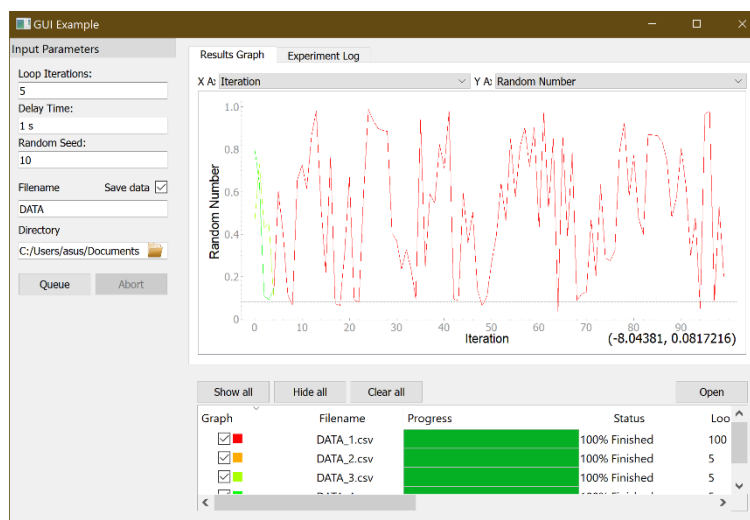
نقاط ضعف:

یک ضعف مشهودی که در این کتابخانه دیده می‌شود، الگوی طراحی^{۴۴} کد ضعیف آن است. توسعه‌دهندگان اصلی این کتابخانه دستورالعمل یا الگوی طراحی کد برای مشارکت‌کنندگان^{۴۵} علاقه‌مند به توسعه پروژه‌های متن باز ارائه داده‌اند ولی مشارکت‌کنندگان در این پروژه بر اساس سلیقه شخصی نام متدها و ساختار کد کلاس‌ها را نوشته‌اند و سخت‌گیری روی کد آن‌ها نشده است. چیزی که از این کتابخانه با این تنوع انتظار می‌رود این است که هر سری و مدل از دستگاه‌ها را بر اساس شرکت سازنده یا کاربرد اندازه‌گیری (مولتی‌متر، منبع تغذیه و...) آن دسته‌بندی کند و برای هر کدام متدهای مشترک تعریف کند. مثلاً دیده می‌شود برای کلاس‌های دستگاه‌های مولتی‌متر، نام متدهای مختلفی برای اندازه‌گیری ولتاژ در نظر گرفته شده است.

⁴⁴ Design pattern

⁴⁵ Contributors

فصل دوم: مفاهیم پایه و کارهای مرتبط



شکل ۱۴ محیط برنامه Pymasure به صورت گرافیکی در حال نمایش اندازه‌گیری‌ها است

۲-۲-۱-۲ دیگر کتابخانه‌ها

برخی کتابخانه‌های دیگر برای ایجاد ارتباط سطح بالاتر با دستگاه‌ها توسعه یافته‌اند. با این حال، این کتابخانه‌ها معمولاً قابلیت‌های محدودتری داشته و تنها تعداد کمتری از دستگاه‌ها را پشتیبانی می‌کنند. همچنین هدف آن‌ها اغلب خاص و محدود به یک کاربرد آزمایشگاهی بوده و در مقایسه با Pymasure کاربران کمتری دارند.

جدول ۵ مقایسه ابزارهای کنترل سخت‌افزار

کتابخانه	وضعیت توسعه	پشتیبانی از دستگاه‌ها	طراحی کد	رابط گرافیکی	مستندسازی	تجربه و نکات عملی
Pymasure	فعال	گسترده	خوب ولی ضعف در انسجام الگوها و نام متدها	دارد (فقط نمودار)	کامل	بهترین گزینه برای پروژه فعلی
Testgear ⁴⁶	نیمه‌فعال	محدود	ساده	ندارد	ندارد	از کد نوشته شده برای دستگاه مولتی‌متر استفاده شده در این پروژه الگو گرفته شد
py-lab-hal ⁴⁷	نیمه‌فعال	محدود	با وجود دستگاه‌های محدود، حالت انتزاعی بهتری نسبت	ندارد	مختصر	طراحی تمیز، نیازمند افزودن درایورهای بیشتر

⁴⁶ <https://github.com/PhilippCo/testgear>

⁴⁷ <https://github.com/google/py-lab-hal>

فصل دوم: مفاهیم پایه و کارهای مرتبط

			Pymeasure ایجاد کرده است			
Instrumental ⁴⁸	نیمه فعال	محدود و خاص (دستگاه‌های دوربین، لیزر)	ساده	ندارد	مختصر	طراحی شده برای نیاز خاص آزمایشگاهی

۲-۱-۳ LabView

نرم افزار LabView که توسط شرکت National Instrument توسعه یافته است، در کنار تجهیزات اندازه گیری ساخت همین شرکت عرضه می شود و مجموعه ای از ابزارها برای اندازه گیری و تست فراهم می آورد. با وجود این، درایورها و پروتکل های ارتباطی موجود در آن عمدتاً محدود به تجهیزات تولید شده توسط همین شرکت NI هستند و هرچند امکان اتصال به تجهیزات دیگر نیز وجود دارد ولی این کار با چالش هایی همراه است.

این نرم افزار متن-بسته بوده و تنها بر روی ویندوز اجرا می شود و در این پروژه که ارتباط تجهیزات از طریق درایور linux-gpib روی رزبری پای برقرار می شد، اتصال مستقیم امکان پذیر نبود. هرچند LabView می تواند از Visa بهره بگیرد، اما به دلیل وابستگی آن به ویندوز و ناسازگاری با رزبری پای مجهز به درایور linux-gpib، برقراری ارتباط بین این دو سیستم ممکن نبود.

۲-۲-۲ ابزارهای اجرای آزمون و خودکارسازی اندازه گیری

در این بخش به بررسی چند ابزار متن باز و تجاری می پردازیم. ابزارهای نوشتن توالی آزمون در لایه ای بالاتر از لایه ای انتزاعی فراخوانی دستورات و کنترل دستگاه ها است. کاربر سامانه به طور مستقیم با این بخش تعامل دارد و تمام اندازه گیری های متوالی و خودکار خودش را باید در این بخش تعریف، اجرا و پایش کند. هدف از این ارزیابی، شناسایی نقاط قوت، ضعف ها و قابلیت های کلیدی هر یک از این راهکارها است تا بتوان با دیدی جامع، بهترین ابزار را برای اهداف پروژه انتخاب کرده و یا از تجربیات موجود برای توسعه راهکار اختصاصی بهره برد.

⁴⁸ <https://github.com/mabuchilab/Instrumental>

فصل دوم: مفاهیم پایه و کارهای مرتبط

روش ارزیابی

برای بررسی هر یک از ابزارها، به مستندات ابزارها مراجعه کردیم و با ویژگی‌ها و قابلیت‌های آن‌ها آشنا شدیم. سپس آن‌هایی را که پتانسیل احتمال استفاده در پروژه را داشتند به صورت عملی یک پیاده‌سازی اولیه و پروتوتایپینگ انجام می‌دادیم. در نتیجه‌ی انجام پروتوتایپینگ محدودیت‌ها و نقاط قوت آن‌ها بیشتر شناخته می‌شد.

معیارهای ارزیابی

- برای هر یک از ابزارها به طور عمده چند معیار را مورد بررسی قرار دادیم:
- انعطاف‌پذیری برای توسعه و افزودن قابلیت‌های دلخواه به ابزار
 - راحتی کار با ابزار هم برای توسعه‌دهنده و هم برای کاربر سامانه
 - قابلیت گزارش‌گیری در قالب‌های مختلف (مانند pdf) و در اختیار قرار دادن نتایج اجرای تست‌ها
 - تعامل پویا در حین اجرای تست با کاربر سامانه
 - توانایی به کارگیری از ساختارهای کنترلی (شرط، حلقه)
 - سازگاری با زبان پایتون و API: برای پیاده‌سازی یک محیط گرافیکی یکپارچه نیاز به لاگ روند اجرای آزمون داریم.
 - دارای رابط گرافیکی

جدول ۶ مقایسه ابزارهای اجرای آزمون

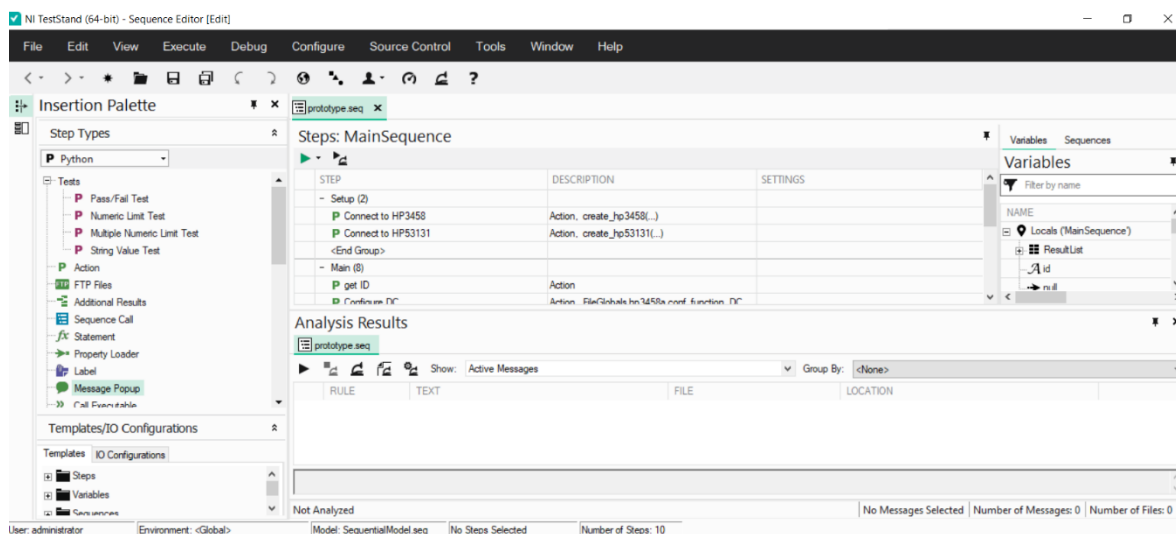
معیار ارزیابی	TestStand	OpenHTF	Flojoy	Robot Framework
افزودن قابلیت‌های بیشتر	کم - به خاطر متن بسته بودن	دارد	دارد (افزودن نودهای سفارشی)	دارد (کتابخانه‌ها و افزونه‌های متعدد)
راحتی کار با ابزار	متوسط (دارای محیط گرافیکی ولی همراه با چالش)	متوسط (نیاز به دانش پایتون)	زیاد (محیط گرافیکی no-code)	زیاد (افزونه VSCode)
قابلیت گزارش‌گیری	دارد	محدود (در اختیار گذاشتن لاگ‌ها با فرمت JSON)	ندارد	دارد (با امکان تحلیل آماری)
توانایی استفاده از ساختارهای کنترلی (شرط، حلقه)	دارد	ندارد	دارد	دارد

فصل دوم: مفاهیم پایه و کارهای مرتبط

داشتن API برای دسترسی از یک برنامه خارجی	دارد ولی سخت و چالشی	ندارد	ندارد	دارد
دارای رابط گرافیکی	دارد	دارد (دو رابط گرافیکی رایگان و غیررایگان)	دارد	ندارد

۲-۲-۱-۱ TestStand

یک نرم‌افزار تجاری و متن‌بسته تحت ویندوز از شرکت National Instruments است که برای طراحی و اجرای تست‌های خودکار توسعه داده شده است. در این نرم‌افزار کاربر به کمک رابط گرافیکی برنامه، گام‌های تست را تعریف و اجرا می‌کند. تعریف تست‌ها به کمک اجرای یک کد پایتون، LabView، کد C و Net قابل انجام است.



شکل ۱۵ تصویری از محیط برنامه TestStand

قابلیت‌های نرم‌افزار:

- این ابزار امکان اجرای سکانس‌های^{۴۹} تست به صورت سلسله‌مراتبی (فراخوانی یک سکانس از داخل دیگری) را فراهم می‌کند.

⁴⁹ Sequence

فصل دوم: مفاهیم پایه و کارهای مرتبط

- به کاربر اجازه می‌دهد تا سکانس‌های مختلف را به صورت موازی و با بهره‌گیری از چنددخی^{۵۰} اجرا کند و برای جلوگیری از دسترسی همزمان به منابع مشترک از قفل‌ها استفاده کند.
- از ساختارهای کنترلی مانند شرط و حلقه پشتیبانی می‌کند.
- می‌تواند توابع پایتون را فراخوانی کرده، پارامترهای ورودی را تنظیم کند و خروجی تابع را برای استفاده در گام‌های^{۵۱} بعدی ذخیره کند.
- حین اجرای تست امکان دریافت ورودی از کاربر را دارد. (مثلاً از کاربر درخواست کند اتصالات سیم‌ها را تغییر دهد و سپس ادامه تست‌ها را اجرا کند)
- امکان تولید گزارش در فرمت PDF را دارد.
- TestStand یک API برای موتور اجرایی خود ارائه می‌دهد که به کمک آن می‌توان از طریق کدهای پایتون، فرآیند اجرای تست‌ها را به صورت برنامه‌نویسی شده و بدون نیاز به محیط گرافیکی کنترل کرد.

محدودیت‌ها:

با وجود تمام قابلیت‌ها، یک ضعف مشخص در نحوه تعامل این ابزار با توابع پایتون مشاهده شد. هنگامی که یک تابع پایتون دارای پارامترهایی با مقدار پیش‌فرض^{۵۲} است، TestStand کاربر را مجبور به مقداردهی تمامی پارامترها می‌کند و از مقدار پیش‌فرض تعریف‌شده در کد چشم‌پوشی می‌کند. این محدودیت می‌تواند فرآیند پیکربندی تست‌ها را کمی کندتر نماید. تعریف و مقداردهی گام‌های تست در این محیط گرافیکی برای کاربر فرآیندی کند است. حتی برای توسعه‌دهنده مراحل انجام این فرآیند فراموش می‌شد!

این یک برنامه متن‌بسته است و خیلی دست برنامه‌نویس را برای افزودن قابلیت‌ها باز نمی‌گذارد. در نیازمندی‌های ما یک محیط گرافیکی برای پایش لحظه‌ای مقادیر اندازه‌گیری شده توسط دستگاه‌ها تعریف شده است ولی از آنجایی که این برنامه قابلیت گسترش ندارد، هم کاربر و هم برنامه‌نویس محدود به استفاده تنها از این برنامه هستند. این نرم‌افزار یک API تحت ویندوز هم ارائه می‌دهد ولی مستندات و کار با این API تقریباً سخت بود.

⁵⁰ Multi-threading

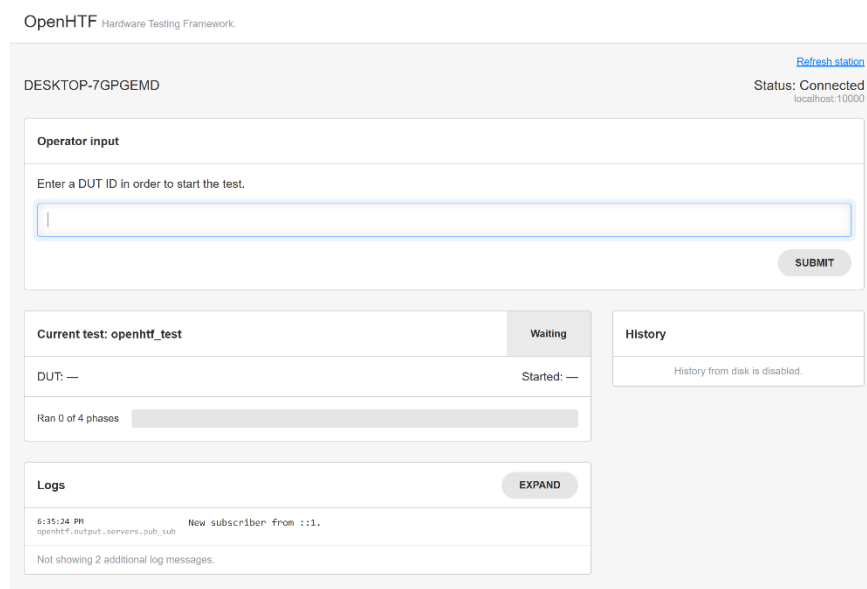
⁵¹ Step

⁵² Default Value

فصل دوم: مفاهیم پایه و کارهای مرتبط

OpenHTF ۲-۲-۲-۲

OpenHTF⁵³ یک چارچوب متن‌باز پایتونی است که در ابتدا توسط گوگل برای خودکارسازی تست‌های سخت‌افزاری توسعه یافته است. توسعه‌ی تست‌ها مشابه کتابخانه‌های Pytest و unittest در قالب توابع پایتونی صورت می‌گیرد. در مفاهیم این چارچوب به هر یک از گام‌ها یا تست‌ها، فاز⁵⁴ گفته می‌شود که هر کدام از توابع پایتون یک فاز تست در نظر گرفته می‌شود.



شکل ۱۶ تصویری از محیط گرافیکی اجرای فازها

امکانات چارچوب:

این چارچوب با هدف تست تجهیزات سخت‌افزاری توسعه یافته است و نام کامل آن Open Hardware Test Framework است. از این جهت قابلیت‌هایی که وجود دارد برای تست تجهیزات سخت‌افزاری کاربردی است. در هر یک از فازهای تست امکان گزارش یک یا چند مقدار اندازه‌گیری شده به همراه یکای⁵⁵ آن‌ها وجود دارد. برای ارزیابی pass/fail فاز تست بررسی‌های مختلفی روی این مقادیر ارائه می‌دهد. از جمله بررسی برابری مطلق یک مقدار اندازه‌گیری شده با یک عدد، قرار گرفتن مقدار در یک بازه مشخص، یا برابری آن با یک درصد خطای مجاز.

⁵³ <https://github.com/google/openhtf>

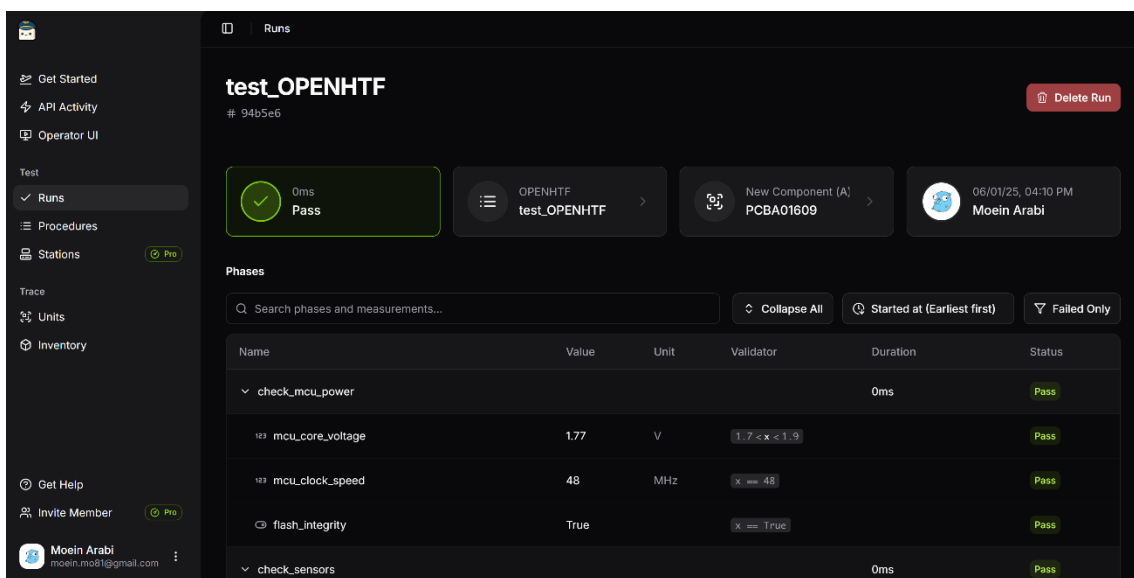
⁵⁴ Phase

⁵⁵ Unit

فصل دوم: مفاهیم پایه و کارهای مرتبط

این فریورک با ارائه یک رابط گرافیکی به صورتی لحظه‌ای امکان پایش روند اجرای فازها، مقادیر اندازه‌گیری شده و لاگ‌ها را می‌دهد. از آنجایی که این چارچوب با هدف تست سخت‌افزارها توسعه یافته است امکان درخواست ورودی از کاربر حین اجرای تست را فراهم می‌کند. مثلاً اگر رنگ LED روی دستگاه تغییر کرد کاربر روی یک دکمه کلیک کند و تست پاس شود.

یک رابط گرافیکی پیشرفته‌تر به نام TofuPilot وجود دارد. می‌توان به کمک API (نه کاملاً رایگان!) از سرورهای آماده برای به کارگیری این رابط گرافیکی استفاده کنیم یا آن را در قالب یک کانتینر داکر بالا بیاوریم. خوبی این رابط گرافیکی این است که تاریخچه اجرای تست‌ها و لاگ‌ها برای همیشه ذخیره می‌شوند و داده‌های تحلیلی بیشتری از نتایج تست‌ها می‌توان به دست آورد. ولی ضعف این رابط گرافیکی این است که متن‌باز نیست و استفاده از آن نیازمند خرید لایسنس است.



شکل ۱۷ تصویری از رابط گرافیکی پیشرفته‌تر با نام TofuPilot

نقاط ضعف:

نوشتن تست‌ها تنها به زبان پایتون امکان‌پذیر است. به همین خاطر کاربر سامانه باید به زبان پایتون آشنایی داشته باشد تا بتواند تست‌های جدید بنویسد. فازها به صورت توابع پایتون پیاده‌سازی می‌شوند و ترتیب اجرای فازها از قبل مشخص

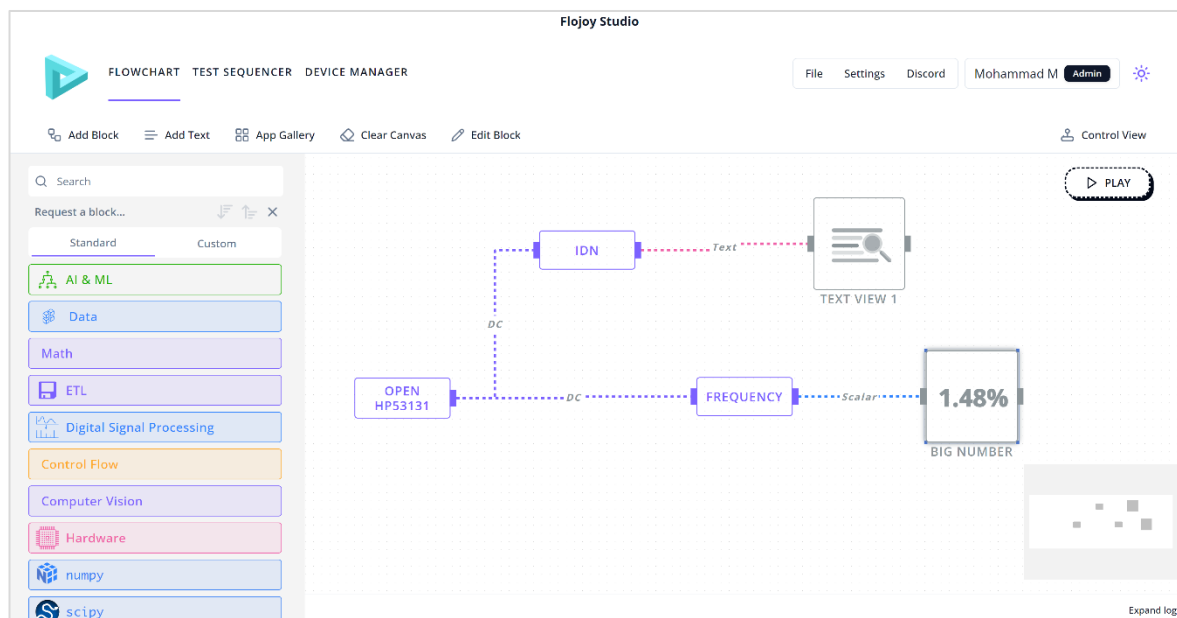
فصل دوم: مفاهیم پایه و کارهای مرتبط

می‌شود؛ به عبارت دیگر این چارچوب ایستا^{۵۶} است، یعنی تمام فازهای تست باید پیش از شروع اجرا تعریف شوند و امکان افزودن یا حذف تست به صورت پویا^{۵۷} در حین اجرا وجود ندارد.

بزرگ‌ترین ضعف این چارچوب، نداشتن ساختارهای کنترلی مانند شرط و حلقه است. طبق نیازمندی‌ها باید امکان تعریف ساختار شرطی و حلقه‌ای را پیاده‌سازی می‌کردیم. مثلاً بر اساس یک مقدار اندازه‌گیری شده یا pass/fail شدن تست‌های قبلی تصمیم‌گیری شود.

Flojoy ۲-۲-۳

^{۵۸}Flojoy با استفاده از FastAPI در بک‌اند و React در فرانت‌اند توسعه یافته است و یک رابط کاربری گرافیکی مدرن برای کنترل تجهیزات ارائه می‌دهد. Flojoy از دو بخش اصلی تشکیل شده است: بخش Flowchart که به کاربر اجازه می‌دهد تا فرآیندهای اندازه‌گیری و پردازش داده را به صورت بصری و با اتصال بلوک‌های عملکردی به یکدیگر طراحی کند، و بخش Test Sequencer که برای اجرای تست‌های از پیش تعریف‌شده به کار می‌رود. کاربر با بارگذاری یک فایل پایتون در قالب Pytest یا Robot روند اجرای آن‌ها را در این محیط گرافیکی مشاهده می‌کند.



شکل ۱۸ محیط برنامه Flojoy و یک نمونه‌سازی برای اندازه‌گیری فرکانس به کمک بخش Flowchart

^{۵۶} Static

^{۵۷} Dynamic

^{۵۸} <https://github.com/flojoy-ai/studio>

فصل دوم: مفاهیم پایه و کارهای مرتبط

امکانات:

بخش Flowchart یک محیط بدون کد⁵⁹ برای کاربر است. این قسمت مانند ابزار مشابه n8n، یک ساختار مبتنی بر فلوچارت و جریان داده ارائه می‌دهد. در این محیط، کاربر با قرار دادن نودها⁶⁰ بر روی صفحه و اتصال خروجی یک نود به ورودی نود دیگر، یک خط لوله پردازشی برای اجرای دستورات و اندازه‌گیری‌ها ایجاد می‌کند. این نرم‌افزار شامل مجموعه‌ای از نودهای پیش‌فرض برای انجام عملیات مختلف است و همچنین به توسعه‌دهندگان اجازه می‌دهد تا نودهای سفارشی خود را برای افزودن قابلیت‌های جدید، ایجاد کنند. نودهای پیش‌فرض در دسته‌بندی‌های زیر قرار می‌گیرند:

- ایجاد ساختارهای کنترلی مانند شرط و حلقه
- نمایش نموداری و بصری‌سازی داده‌ها
- اتصال به تجهیزات اندازه‌گیری پشتیبانی‌شده
- انجام عملیات تحلیل و پردازش داده

محدودیت‌ها:

بررسی تاریخچه کامیت‌ها نشان داد که پروژه در چند ماه اخیر فعالیتی نداشته است که این موضوع، آینده پشتیبانی و رفع اشکالات احتمالی آن را مورد ابهام قرار می‌دهد. علاوه بر این، مستندات پروژه برای توسعه نودهای سفارشی⁶¹ بسیار ضعیف بود، به طوری که برای درک عملکرد داخلی نیازمند بررسی مستقیم سورس کد شدیم. عدم امکان فراخوانی یک فلوچارت از درون فلوچارتی دیگر، قابلیت ماژولار کردن و استفاده مجدد از منطق‌های تکراری را از بین می‌برد.

بزرگترین ضعف دیگر، عدم تعامل یکپارچه بین دو بخش اصلی یعنی Flowchart و Test Sequencer بود. این دو ماژول به صورت کاملاً مجزا عمل می‌کردند و هیچ راهکار مستقیمی برای اجرای یک فلوچارت بصری به عنوان یک گام در یک توالی تست وجود نداشت. کاربر نمی‌تواند طراحی بصری خود را به یک تست قابل اجرا تبدیل کند و همچنین هیچ امکانی برای دریافت گزارش از خروجی اجرای تست‌ها در محیط گرافیکی تعبیه نشده بود.

⁵⁹ No-Code

⁶⁰ Nodes

⁶¹ Custom

HILSTER ۲-۲-۲-۴

چارچوب HILSTER⁶² یک محصول تجاری است که متن‌باز نبوده، اما در دو نسخه Community و تجاری عرضه می‌شود. یکی از ویژگی‌های قابل توجه HTF، انعطاف‌پذیری آن در نحوه تعریف آزمون‌هاست؛ این ابزار به مهندسان اجازه می‌دهد تا تست‌ها را هم به صورت روبه‌ای و در قالب توابع پایتون (مشابه رویکرد Pytest یا OpenHTF) و هم به سبک BDD و با استفاده از یک زبان اسکریپت‌نویسی سطح بالا (مشابه Robot Framework) پیاده‌سازی کنند.

یکی از قابلیت‌های منحصربه‌فرد این چارچوب، امکان مدیریت و ردیابی نیازمندی‌ها⁶³ بود. در این سیستم، کاربر می‌توانست لیستی از نیازمندی‌های پروژه را تعریف کرده و هر آزمون را به یک یا چند نیازمندی مشخص نسبت دهد. در نهایت، سیستم گزارش‌هایی از میزان پوشش نیازمندی‌ها توسط تست‌ها ارائه می‌داد. با وجود داشتن یک رابط کاربری گرافیکی برای نمایش گزارش‌ها، ماهیت متن‌بسته بودن این ابزار باعث شده است تا جامعه کاربری کوچک‌تر و منابع پشتیبانی محدودتری نسبت به جایگزین‌های متن‌باز داشته باشد.

Robot ۲-۲-۲-۵

یک چارچوب متن‌باز تحت پایتون است که امکان نوشتن تست‌ها را در قالب اسکریپت با زبان اختصاصی خود فراهم می‌کند. این چارچوب مبتنی بر پایتون است و بر خلاف چارچوب‌های دیگر (مانند OpenHTF)، تست‌ها در یک زبان اختصاصی و سطح بالایی دیگر نوشته می‌شوند. این یک مزیت برای افرادی است که دانش کافی از برنامه‌نویسی زبان پایتون ندارند.

کاربر در یک فایل با پسوند robot، تست‌ها را می‌نویسد. هر فایل می‌تواند شامل یک یا چند Test case باشد و هر Test case مجموعه‌ای از کلمات کلیدی⁶⁴ را فراخوانی می‌کند. به ازای هر یک از کلمات کلیدی یک تابع پایتون قرار دارد.

امکانات:

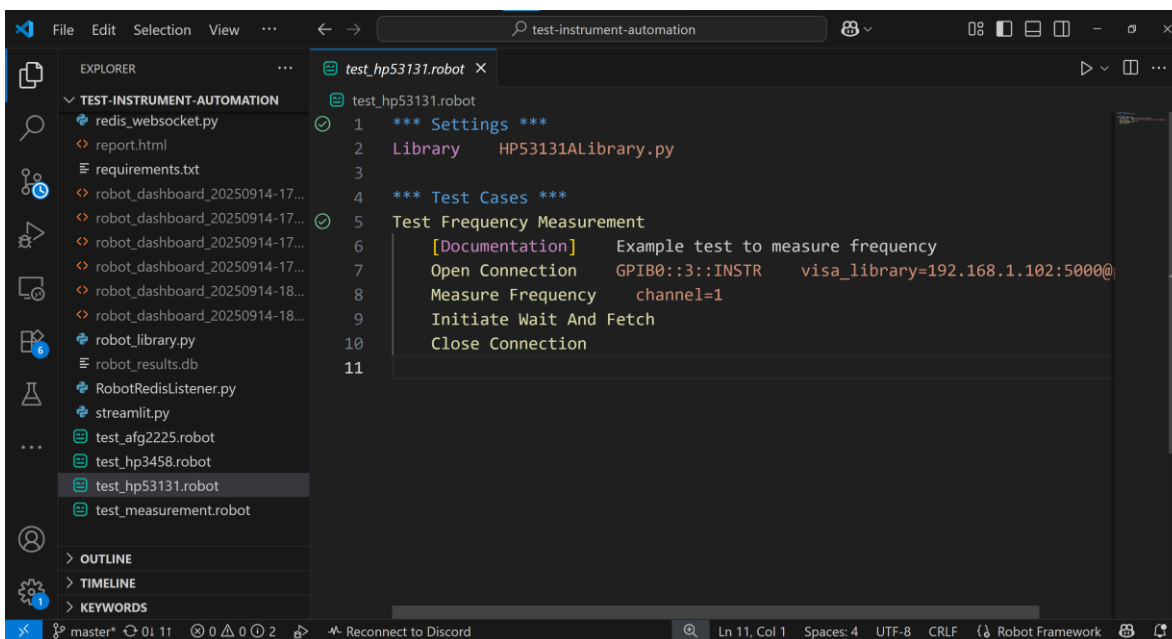
⁶² <https://hilster.io>

⁶³ Requirements Traceability

⁶⁴ Keyword

فصل دوم: مفاهیم پایه و کارهای مرتبط

با ارائه افزونه^{۶۵} اختصاصی برای ویرایشگر VS-Code، تجربه کاربری بسیار برتری را فراهم می‌کند. قابلیت‌هایی نظیر تکمیل خودکار کد^{۶۶}، نمایش مستندات توابع و بررسی آئی سینتکس، فرآیند توسعه تست را به شکل چشمگیری ساده‌تر و کم‌خطاتر می‌سازد.



شکل ۱۹ تصویری از محیط برنامه توسعه VSCode که افزونه Robot نصب گردیده است و در حال نمایش یک فایل تست است

Robot Framework به صورت داخلی از ساختارهای قدرتمندی مانند شرط، حلقه، بلوک‌های Setup و Teardown و تعریف کلمات کلیدی^{۶۷} پشتیبانی می‌کند. امکان گزارش گرفتن در قالب فایل HTML، ذخیره‌سازی در دیتابیس و تحلیل آماری روی نتایج تست‌ها دارد.

جامعه کاربری Robot Framework به دلیل متن‌باز بودن، از یک جامعه کاربری بسیار فعال و بزرگ بهره می‌برد. این امر منجر به توسعه کتابخانه‌های جانبی^{۶۸} متعدد و منابع آموزشی گسترده شده است.

^{۶۵} Extention

^{۶۶} Auto-complete

^{۶۷} Keywords

^{۶۸} Third-party

۲-۲-۳ ابزارهای رابط گرافیکی

در این پروژه برای پیاده‌سازی سریع‌تر و حفظ تمرکز بر منطق اصلی برنامه، از کتابخانه‌های گرافیکی پایتون استفاده شد.

۲-۲-۳-۱ Dash

Dash یک چارچوب پایتونی برای ساخت داشبوردهای وب است که به توسعه‌دهنده اجازه می‌دهد با نوشتن کد پایتون، صفحات تعاملی ایجاد کند. محدودیت‌ها:

مهم‌ترین چالش، ترکیب منطق ساختاری وب (HTML/CSS) با کد پایتون است که منجر به کاهش خوانایی و افزایش پیچیدگی کد، به ویژه در پروژه‌های بزرگ می‌شود. این رویکرد، سرعت نمونه‌سازی اولیه را در مقایسه با ابزارهای ساده‌تری مانند Streamlit به شکل قابل توجهی کاهش می‌دهد. علاوه بر این، ظاهر پیش‌فرض Dash فاقد جذابیت مدرن است و برای رسیدن به یک رابط کاربری قابل قبول، تقریباً همیشه نیاز به اعمال CSS خودمان وجود دارد. در نهایت، مزیت اصلی Dash یعنی قابلیت شخصی‌سازی بالا، در عمل آن را در یک جایگاه میانی نامناسب قرار می‌دهد؛ برای پروژه‌هایی که به کنترل کامل بر روی رابط کاربری نیاز دارند، استفاده مستقیم از چارچوب‌های وب مانند React گزینه منطقی‌تری است.

۲-۲-۳-۲ Streamlit

Streamlit یک چارچوب متن‌باز و سبک در پایتون است که با هدف اصلی تسهیل و تسریع فرآیند ساخت اپلیکیشن‌های وب تعاملی، به ویژه برای پروژه‌های داده‌محور طراحی شده است. فلسفه اصلی این ابزار، تمرکز بر سادگی و سرعت نمونه‌سازی^{۶۹} اولیه است و به توسعه‌دهندگان اجازه می‌دهد تا با دانش پایتون، به سرعت ایده‌های خود را به یک رابط کاربری گرافیکی کاربردی تبدیل کنند.

نقطه قوت آن در سادگی API و سرعت بالای توسعه آن است. توسعه‌دهندگان می‌توانند با چند خط کد، یک رابط کاربری تعاملی بسازند، بدون آنکه نیازی به درگیر شدن با پیچیدگی‌های HTML، CSS یا JavaScript داشته باشند. رابط کاربری پیش‌فرض این چارچوب مدرن و قابل قبول است و نیاز به شخصی‌سازی را برطرف می‌کند. علاوه بر این، Streamlit از قابلیت توسعه‌پذیری بالایی برخوردار است و به کاربران اجازه می‌دهد تا کامپوننت‌های سفارشی خود را ایجاد کنند. این کامپوننت‌ها

^{۶۹} Prototyping

فصل دوم: مفاهیم پایه و کارهای مرتبط

که می‌توانند با هر فناوری وبی مانند React توسعه داده شوند، از طریق یک iframe در برنامه اصلی رندر شده و به کمک قالب‌های آماده‌ای که توسط تیم Streamlit ارائه شده، فرآیند ساخت API پایتونی برای آن‌ها نیز تسهیل شده است.

۴-۲-۲ ابزارهای ارتباطی

در این بخش به بررسی ابزارها و کتابخانه‌های موجود برای ارسال دستورات به دستگاه و دریافت داده‌ها از آن‌ها می‌پردازیم. تجهیزات آزمایشگاهی از پروتکل‌های مختلفی مانند GPIB، USB، Ethernet و CAN استفاده می‌کنند. وجود لایه‌های انتزاعی و سطح بالا برای پوشش دادن هر نوع پروتکل اهمیت دارد.

در این پروژه، کتابخانه PyVISA به عنوان کتابخانه اصلی برای برقراری ارتباط با تجهیزات GPIB انتخاب شد. این کتابخانه با فراهم آوردن یک لایه انتزاعی قدرتمند، پیچیدگی‌های کار با درایورهای سطح کرنل و پروتکل‌های لایه پایین را پنهان کرده و یک رابط یکپارچه برای ارسال و دریافت فرمان‌ها ارائه می‌دهد. یکی از مزایای کلیدی PyVISA، پشتیبانی آن از پروتکل‌های ارتباطی متنوعی نظیر سریال و TCP است که انعطاف‌پذیری بالایی را فراهم می‌آورد. علاوه بر این، برای تحقق معماری توزیع‌شده سیستم، از کتابخانه مکمل PyVISA-proxy بهره گرفته شد. این ابزار با گسترش قابلیت‌های PyVISA، امکان برقراری ارتباط و کنترل تجهیزات از طریق شبکه را ممکن می‌سازد و به اجزای مختلف سیستم اجازه می‌دهد تا به صورت غیرمتمرکز با سخت‌افزار در تعامل باشند.

۵-۲-۲ ابزارهای شبیه‌سازی

با توجه به ماهیت عملی پروژه و لزوم تعامل با تجهیزات فیزیکی، شبیه‌سازی رفتار دستگاه‌ها امکان کدنویسی و انجام تست‌های اولیه را بدون نیاز به دسترسی دائمی به سخت‌افزار فراهم می‌کند. برای این منظور، از کتابخانه PyVISA-sim به عنوان ابزار اصلی شبیه‌سازی در این پروژه استفاده گردید.

این کتابخانه به جای ارسال دستورات به یک دستگاه واقعی، آن‌ها را به یک موتور شبیه‌ساز مبتنی بر فایل کانفیگ YAML هدایت می‌کند. در این فایل، زوج‌های فرمان-پاسخ^{۷۰} تعریف می‌شود (مثلاً اگر دستور رشته‌ای MEASure:FREQuency? ارسال شد در پاسخ یک عدد به عنوان فرکانس اندازه‌گیری شده برگردان). این رویکرد باعث

⁷⁰ Request-Response

فصل دوم: مفاهیم پایه و کارهای مرتبط

می‌شود کد اصلی برنامه هیچ تغییری نکند و تنها با تغییر رشته اتصال، بتوان بین حالت واقعی و شبیه‌سازی جابجا شد. شبیه‌سازی رفتارهای پیچیده با این ابزار ممکن نیست، اما راهکاری سریع محسوب می‌شود. کتابخانه دیگری به نام PyVISA⁷¹ وجود دارد که به خاطر هدایت دستورات به تابع‌های پایتون از پیش تعریف شده امکان ذخیره حالت دستگاه و شبیه‌سازی رفتارهای پیچیده‌تر را هم می‌دهد.

۲-۳ جمع‌بندی

در این فصل به توضیحات پروتکل GPIB و سیگنال‌های باس آن پرداخته شد. دیدیم دستورات تجهیزات اندازه‌گیری از استاندارد SCPI بهره می‌برند که ساختار دستورات مشابهی دارند. با مبدل Prologix که پروتکل Ethernet را به GPIB تبدیل می‌کند آشنا شدیم. مطالعات انجام شده روی چند مدل از دستگاه‌های اندازه‌گیری شامل مولتی‌متر HP-3458A و شمارنده HP-53131A که در این پروژه استفاده شده بود، ذکر شد. در نهایت چند ابزار و کتابخانه که بخشی یا مشابه آن چه در این پروژه می‌خواهیم پیاده‌سازی کنیم را بررسی و مقایسه کردیم. در فصل بعدی به جزئیات معماری و پیاده‌سازی انجام شده می‌پردازیم. ابتدا تصویری کلی از بخش‌های مختلف سیستم با مدل C4 ترسیم خواهد شد، سپس از روی آن به توضیح جزئیات پیاده‌سازی پیاده‌سازی می‌پردازیم.

⁷¹ <https://github.com/microsoft/pyvisa-mock>

فصل سوم: روش پیشنهادی و نتیجه‌گیری

۳-۱ مقدمه

در این بخش، رویکرد بالا-به-پایین اتخاذ شده و نحوه پیاده‌سازی هر یک از مولفه‌های کلیدی معماری، از رابط کاربری گرافیکی و موتور تست گرفته تا لایه‌های انتزاعی سخت‌افزار و مبدل GPIB، به صورت مجزا تشریح خواهند شد. در این قسمت، فناوری‌های به کار رفته، کتابخانه‌های مورد استفاده و تصمیمات مهندسی مورد بحث قرار می‌گیرند. در ادامه، رویه ارزیابی و اعتبارسنجی ارائه می‌گردد که در آن به نحوه صحنه‌سنجی عملکرد سامانه از طریق سناریوهای آزمون واقعی پرداخته می‌شود. در بخش پایانی، نتایج و تحلیل ارائه شده و عملکرد سامانه بر اساس شاخص‌های کلیدی مانند پایداری ارتباط، سرعت اجرای آزمون‌ها و توسعه‌پذیری سیستم ارزیابی خواهد شد.

رویکرد کلی این فصل، ارائه یک گزارش فنی تکرارپذیر از فرآیند توسعه است تا خواننده نه تنها با محصول نهایی، بلکه با چالش‌های فنی و راهکارهای به کار رفته در طول مسیر نیز به خوبی آشنا شود.

۳-۲ ساختار روش پیشنهادی

در این بخش، به تشریح جزئیات فنی و پیاده‌سازی معماری می‌پردازیم. ابتدا گردش کار^{۷۲} کلی سیستم از دیدگاه کاربر نهایی ترسیم شده است تا یک دید کلی به خواننده‌ی این گزارش ارائه دهد. سپس به کمک مدل C4 جزئیات پیاده‌سازی هر یک از بخش‌های اصلی و ارتباط هر بخش با یکدیگر به تفکیک بررسی خواهد شد.

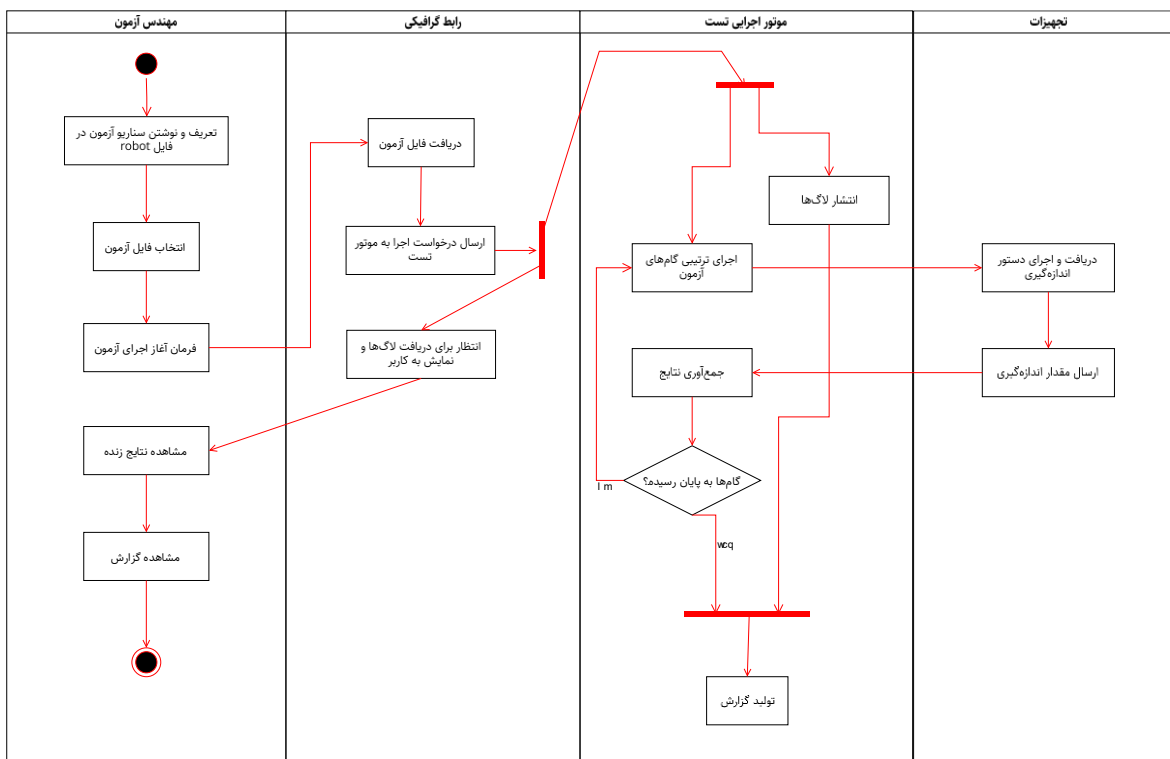
۳-۲-۱ گردش کار کلی سیستم

فرآیند کار با سامانه آزمون خودکار از تعریف یک سناریوی آزمون تا دریافت گزارش نهایی، از یک جریان مشخص پیروی می‌کند. نمودار فعالیت^{۷۳} زیر، این گردش کار را به صورت بصری نمایش می‌دهد.

⁷² Workflow

⁷³ Activity Diagram

فصل سوم: روش پیشنهادی و نتیجه‌گیری



شکل ۲۰ نمودار فعالیت بخش‌های مختلف سیستم از زمان تعریف سناریو آزمون

همانطور که در نمودار مشخص است، فرآیند کلی به گام‌های زیر تقسیم می‌شود:

۱. **تعریف سناریوی آزمون:** مهندس آزمون، توالی گام‌های مورد نیاز برای یک تست را در قالب یک فایل نوشته شده با زبان robot تعریف می‌کند. این فایل شامل اطلاعاتی نظیر دستگاه‌های هدف، توابع مورد فراخوانی، پارامترهای ورودی و معیارهای موفقیت/شکست^{۷۴} هر گام است.
۲. **آغاز آزمون:** کاربر از طریق رابط کاربری گرافیکی، فایل سناریو را انتخاب کرده و فرمان اجرای آزمون را صادر می‌کند.

⁷⁴ Pass/fail

فصل سوم: روش پیشنهادی و نتیجه‌گیری

۳. **اجرا توسط موتور تست:** موتور تست^{۷۵} سناریو را اجرا کرده و گام‌ها را به ترتیب اجرا می‌کند. برای هر گام، موتور تست رویدادهای مربوط به شروع، پایان و نتیجه را در یک کانال رویداد^{۷۶} منتشر می‌کند تا لاگ‌های اجرا شدن برنامه در محیط گرافیکی به صورت لحظه‌ای برای کاربر نمایش داده شود.
۴. **ارتباط با تجهیزات:** موتور تست از طریق لایه‌های انتزاعی سخت‌افزار (PyVISA و Pymmeasure)، دستورات لازم را به تجهیزات اندازه‌گیری ارسال و نتایج را دریافت می‌کند.
۵. **نمایش زنده نتایج:** رابط کاربری گرافیکی که به کانال رویداد متصل است، رویدادها را به صورت زنده دریافت کرده و وضعیت اجرای تست و مقادیر اندازه‌گیری شده را به صورت لحظه‌ای به کاربر نمایش می‌دهد.
۶. **تولید گزارش نهایی:** پس از اتمام تمام گام‌های سناریو، موتور تست یک گزارش جامع از نتایج، شامل وضعیت موفقیت/شکست هر گام، مقادیر اندازه‌گیری شده و زمان‌بندی اجرا، تولید می‌کند.

۳-۲-۲ معماری سطح بالای سیستم

برای تشریح معماری این پروژه از مدل C4 استفاده شده است که یک رویکرد ساده و ساختاریافته برای نمایش معماری نرم‌افزار در سطوح مختلف لایه‌های انتزاعی است. این مدل به ما کمک می‌کند تا با بزرگنمایی در بخش‌های مختلف، به درک عمیق‌تری از ساختار و اجزای سیستم برسیم.

۳-۲-۲-۱ سطح ۱: Context Diagram

Context Diagram، بالاترین سطح از مدل C4 است. این دیاگرام، سیستم نرم‌افزاری ما را در مرکز سیستم خود قرار می‌دهد و تعاملات اصلی آن با کاربران^{۷۷} و سیستم‌های خارجی^{۷۸} را به تصویر می‌کشد. هدف این نمودار، ارائه یک دید کلی از مرزها و ارتباطات اصلی سیستم بدون ورود به جزئیات فنی داخلی است.

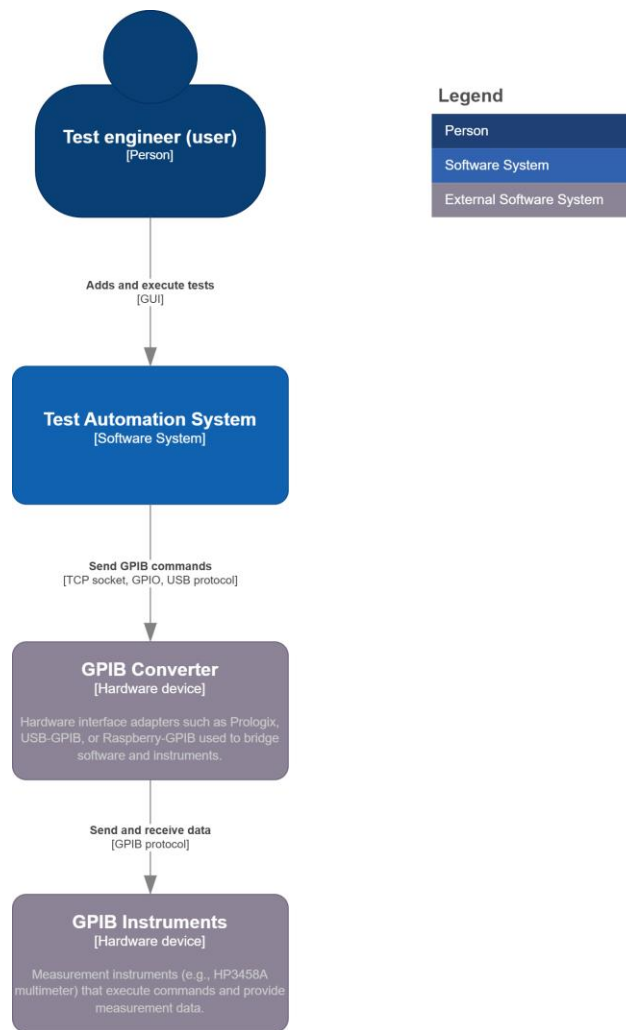
⁷⁵ Test Engine

⁷⁶ Event Broker

⁷⁷ Actors

⁷⁸ External Systems

فصل سوم: روش پیشنهادی و نتیجه‌گیری



شکل ۲۱ سطح Context معماری سیستم با رویکرد مدل C4

همانطور که در دیاگرام بالا مشاهده می‌شود، سیستم از اجزای زیر تشکیل شده است:

- کاربر (Person):
 - مهندس آزمون^{۷۹}: کاربر اصلی سیستم است که از طریق یک رابط کاربری گرافیکی با سامانه تعامل دارد. وظیفه او تعریف، افزودن، اجرا و پایش سناریوهای آزمون برای تجهیزات مختلف است.
- سیستم نرم‌افزاری (Software System):

⁷⁹ Test Engineer

فصل سوم: روش پیشنهادی و نتیجه‌گیری

○ سامانه آزمون خودکار^{۸۰}: این همان سیستم نرم‌افزاری است که در این پروژه طراحی و پیاده‌سازی شده است.

این سامانه به عنوان یک واسطه بین مهندس آزمون و سخت‌افزارهای اندازه‌گیری عمل می‌کند.

• سیستم‌های خارجی (External Systems):

○ مبدل GPIB: یک دستگاه سخت‌افزاری واسطه است که فرمان‌های دریافتی از سامانه نرم‌افزاری (از طریق

پروتکل‌هایی مانند TCP/IP, USB یا GPIO) را به پروتکل استاندارد GPIB ترجمه می‌کند

○ تجهیزات GPIB: دستگاه‌های اندازه‌گیری نهایی مانند مولتی‌متر، شمارنده و... که از طریق پروتکل GPIB

کنترل می‌شوند و داده‌های اندازه‌گیری شده را ارسال می‌کنند.

جریان تعامل اصلی در این دیاگرام به این صورت است که مهندس آزمون از طریق رابط کاربری، آزمون‌ها را به سامانه

آزمون خودکار ارسال می‌کند. سپس، این سامانه فرمان‌های لازم را به مبدل GPIB ارسال کرده و در نهایت، مبدل این

فرمان‌ها را برای کنترل تجهیزات GPIB به کار می‌گیرد.

۳-۳ پیاده‌سازی روش پیشنهادی

در بخش قبلی با کلیات معماری سیستم آشنا شدیم. در این بخش جزئیات هر بخش، کتابخانه‌ها و چارچوب‌های به کار

گرفته برای پیاده‌سازی توضیح داده می‌شود.

۳-۳-۱ سطح ۲: Container Diagram

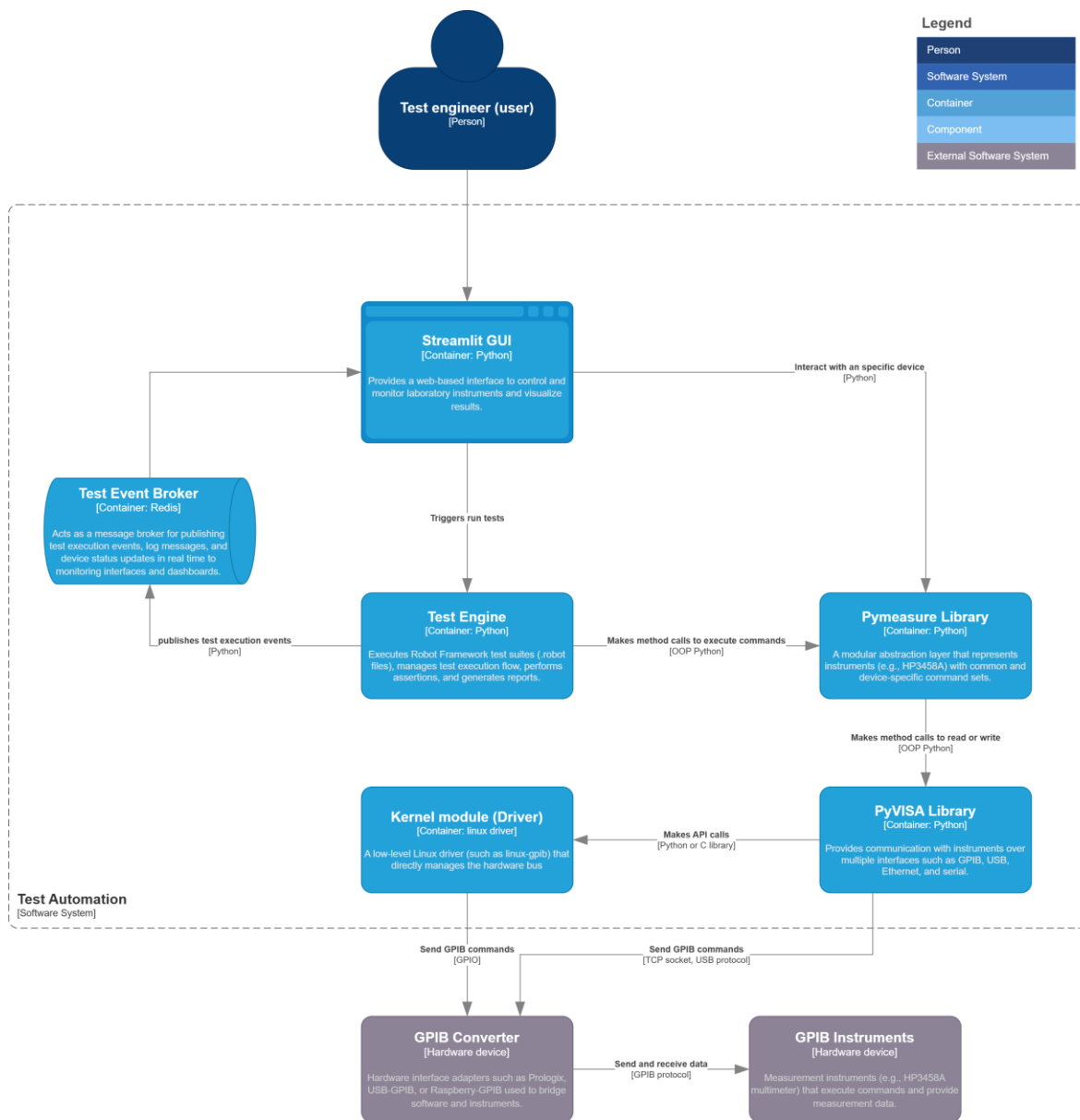
در این سطح، هر یک از این کانتینرها یک واحد مستقل و قابل اجرا هستند، مانند یک برنامه وب، یک سرویس API یا

یک کتابخانه اصلی که فرآیندهای اصلی را مدیریت می‌کند. این دیاگرام به ما کمک می‌کند تا مسئولیت‌های اصلی سیستم را

تقسیم‌بندی کرده و فناوری‌های به کار رفته در هر بخش را مشخص کنیم.

⁸⁰ Test Automation System

فصل سوم: روش پیشنهادی و نتیجه‌گیری



شکل ۲۲ سطح Container معماری سیستم با رویکرد مدل C4

به طور کلی کاربر از طریق رابط گرافیکی با موتور تست تعامل می‌کند. موتور تست از طریق لایه‌های انتزاعی Pymeasure و PyVISA دستورات را به درایور کرنل می‌رساند و در نهایت این دستورات به سخت‌افزار خارجی ارسال می‌شوند. بر اساس نمودار بالا، سامانه آزمون خودکار از کانتینرهای زیر تشکیل شده است:

فصل سوم: روش پیشنهادی و نتیجه‌گیری

۱-۳-۱-۱ رابط کاربری گرافیکی

برای پیاده‌سازی رابط کاربری، از چارچوب Streamlit استفاده شد. دلیل اصلی این انتخاب، قابلیت نمونه‌سازی سریع و یکپارچگی کامل با زبان پایتون بود که به توسعه‌دهنده اجازه می‌دهد بدون نیاز به دانش گسترده از فناوری‌های وب (HTML, CSS, JavaScript)، یک رابط کاربری تعاملی ایجاد کند. این رابط گرافیکی از طریق اتصال به کانال رویداد Redis، به صورت زنده رویدادهای منتشر شده توسط موتور تست را دریافت کرده و وضعیت سیستم را به صورت لحظه‌ای برای کاربر به‌روزرسانی می‌نماید.

۱-۳-۱-۲ موتور تست

برای خودکارسازی فرآیندهای پیکربندی، اندازه‌گیری و تست، پس از بررسی کتابخانه‌های مختلف، به دلیل ساختار اسکرپت نوشتن، قابلیت توسعه بالا و امکانات گزارش‌گیری، Robot Framework انتخاب شد. در این چارچوب کاربر برای خودکارسازی فرآیندها از زبان Robot برای نوشتن تست‌ها استفاده می‌کند. در پایین یک نمونه کد تست کیس Robot آورده شده است که ابتدا اتصال به دستگاه‌ها باز می‌شود، سپس دستگاه Function generator برای خروجی فرکانس ۱۰ هزار هرتز تنظیم می‌شود و دستگاه Counter این مقدار را از ورودی کانال دوم خود اندازه‌گیری می‌کند:

```
*** Test Cases ***
Config Function generator and measure frequency
    Connect Instruments
    Configure Function Generator    frequency=10_000
    Measure Frequency    channel=2
    Initiate Wait And Fetch
    Disconnect Instruments
```

۱-۳-۱-۳ صف پیام

برای ایجاد یک معماری توزیع‌شده از یک صف پیام استفاده شد. زمانی که تست‌ها اجرا می‌شوند چند نوع لاگ منتشر می‌شود. این لاگ‌ها با درجه اهمیت‌های متفاوت از بخش‌های مختلف کد منتشر می‌شوند. برای جمع‌آوری همه این لاگ‌ها از الگوی Publish/Subscribe در Redis کمک گرفته شد.

انواع پیام‌هایی که در صف منتشر می‌شوند شامل موارد زیر هستند:

- شروع و پایان اجرای یک کلمه‌کلیدی

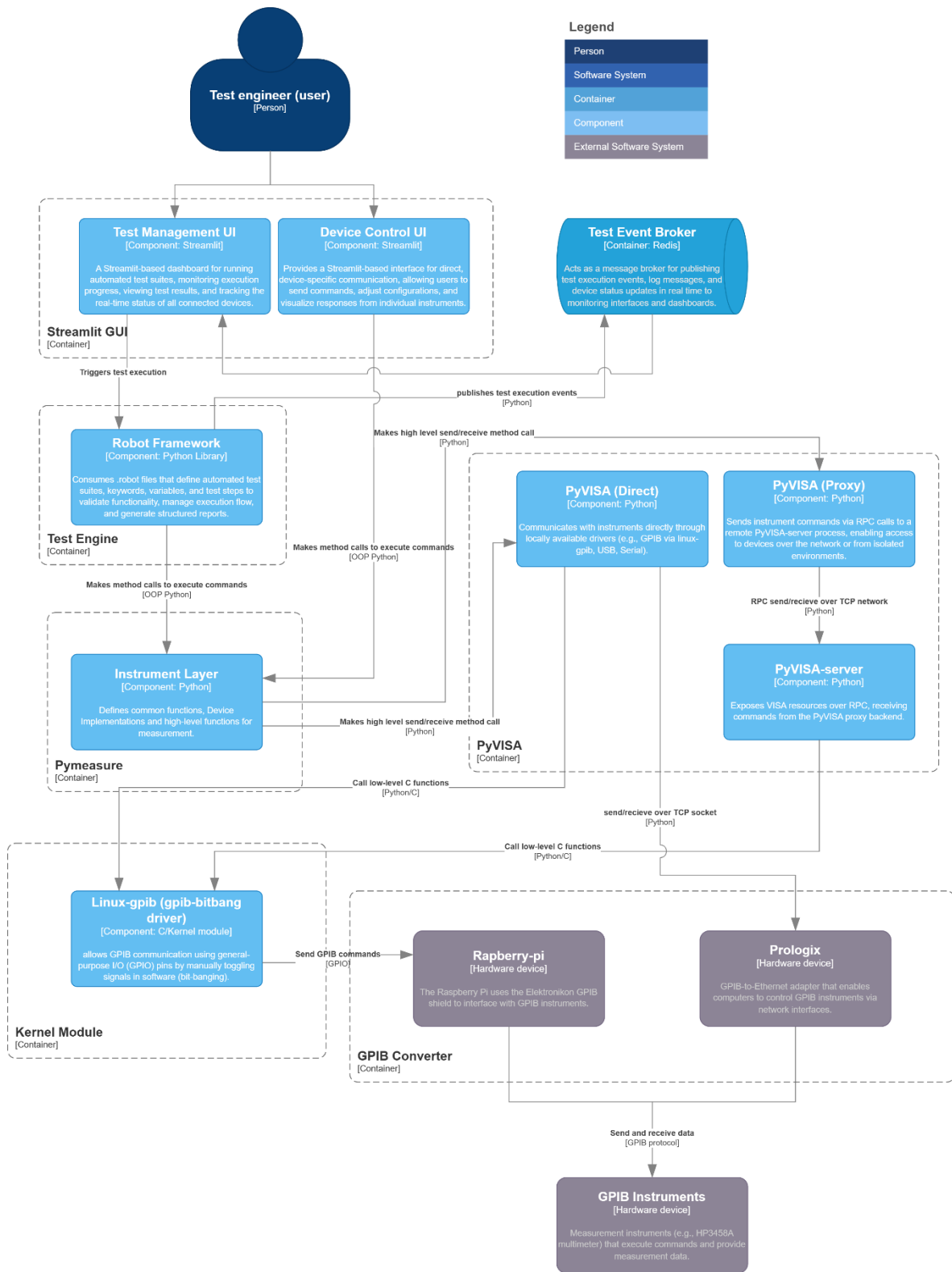
فصل سوم: روش پیشنهادی و نتیجه‌گیری

- هشدارهای خطا در اندازه‌گیری
- مقدارهای اندازه‌گیری شده و وضعیت هر یک از تجهیزات

۲-۳-۳ سطح ۳ و ۴: Component/code Diagram

در این سطح، کانتینرهای اصلی سیستم را به اجزای سازنده‌شان تجزیه می‌کنیم. هر کامپوننت یک گروه منطقی از کدها (مانند یک کلاس یا مجموعه‌ای از توابع) است که یک مسئولیت مشخص را بر عهده دارد. این دیاگرام جزئیات پیاده‌سازی و ساختار داخلی هر بخش از سیستم را روشن می‌کند. در شکل ۲۳ دیاگرام سطح Component قابل مشاهده است.

فصل سوم: روش پیشنهادی و نتیجه گیری



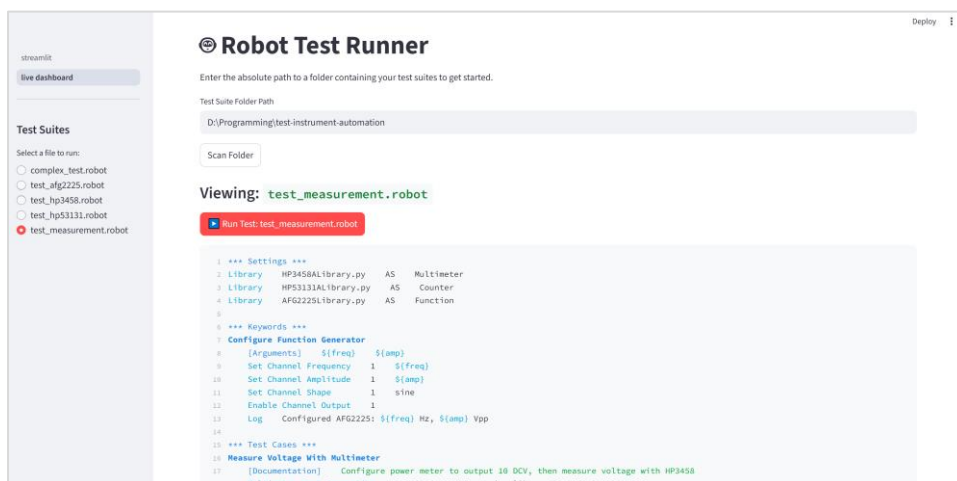
شکل ۲۳ سطح Component معماری سیستم با رویکرد مدل C4

فصل سوم: روش پیشنهادی و نتیجه‌گیری

۳-۲-۱-۳-۲-۱ پایش لحظه‌ای

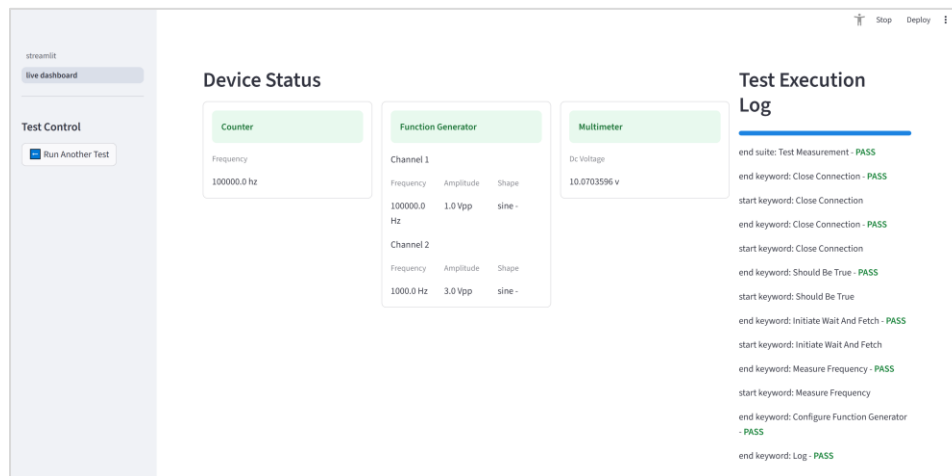
رابط کاربری گرافیکی سیستم شامل بخش‌های مجزا برای انتخاب فایل سناریوی تست و نمایش زنده لاگ‌های اجرا می‌باشد. بخش رابط مدیریت آزمون مسئول نمایش وضعیت اجرای آزمون‌ها و نمایش نتایج لحظه‌ای است. این بخش با صف رویدادها در ارتباط است تا وضعیت لحظه‌ای را دریافت کند.

در شکل ۲۴ صفحه انتخاب فایل تست برای اجرا را نمایش می‌دهد. در این صفحه کاربر یک پوشه از سیستم خود انتخاب می‌کند و برنامه با تشخیص فایل‌های تست موجود آن‌ها را به کاربر نشان می‌دهد. سپس کاربر می‌تواند آن را اجرا کند. در شکل ۲۵ صفحه پایش لحظه‌ای اندازه‌گیری و رویدادها حین اجرای تست است. در این صفحه کاربر همزمان هر سه دستگاه متصل شده و در سمت راست رویدادهای اجرای گام‌های تست را مشاهده می‌کند.



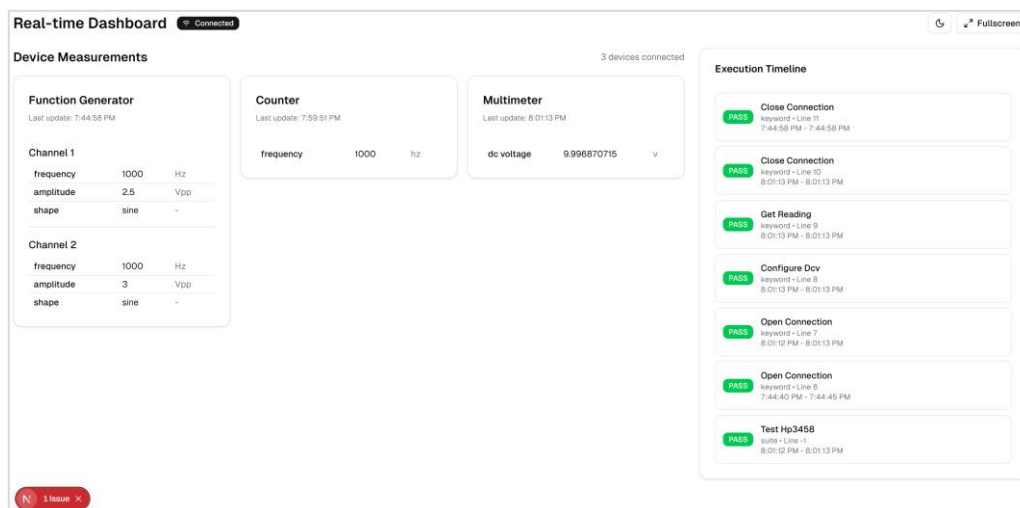
شکل ۲۴ عکس از نمونه‌سازی اولیه محیط رابط گرافیکی برای انتخاب و اجرای فایل تست

فصل سوم: روش پیشنهادی و نتیجه گیری



شکل ۲۵ صفحه پایش لحظه‌ای اندازه‌گیری‌ها و رویدادها حین اجرای آزمون

در شکل ۲۶ یک محیط گرافیکی با فناوری React و به کمک هوش مصنوعی مولد ⁸¹v0 توسعه داده شده است. ارتباط این رابط گرافیکی از طریق web-spocket با سرور برای دریافت رویدادهای لحظه‌ای اتفاق می‌افتد. این محیط گرافیکی برای تحقیق و ارزیابی فناوری React صورت گرفته است و رابط گرافیکی جامع‌تر در این پروژه با فناوری Streamlit توسعه داده شده است.

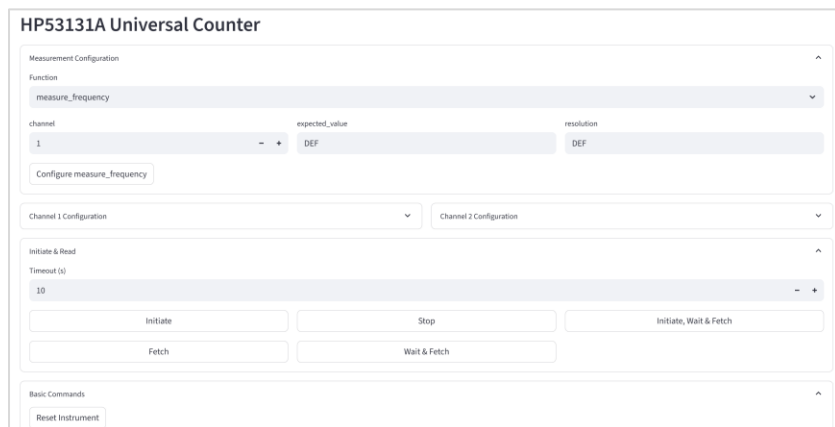


شکل ۲۶ صفحه پایش اندازه‌گیری ساخته شده مبتنی بر فناوری React

⁸¹ <https://v0.dev>

فصل سوم: روش پیشنهادی و نتیجه‌گیری

بخش رابط کنترلی دستگاه به کاربر اجازه می‌دهد تا به صورت دستی به دستگاه‌ها متصل شده، پارامترهای آن‌ها را تنظیم کرده و دستورات کنترلی را مستقیماً برای آن‌ها ارسال کند. در شکل ۲۴ عکسی از صفحه دستگاه شمارنده مشاهده می‌شود.



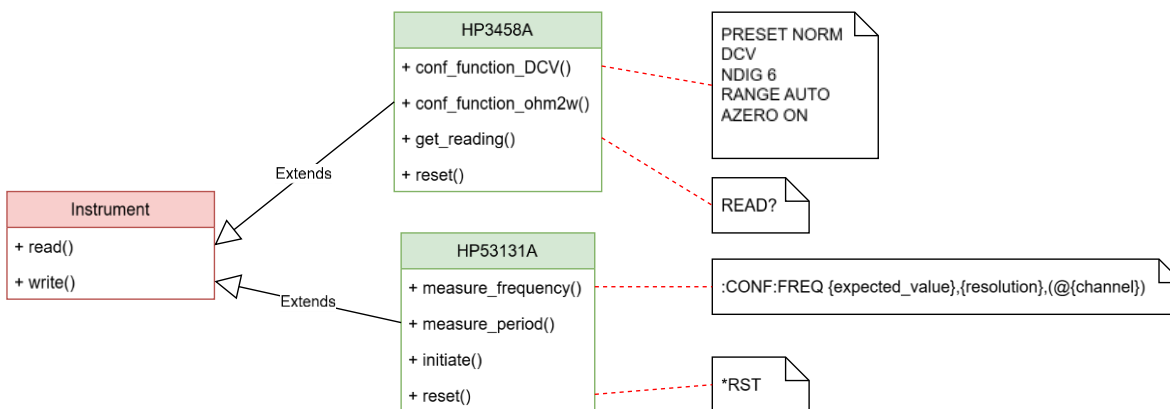
شکل ۲۷ صفحه محیط گرافیکی برای اتصال دستی به یک دستگاه و انجام یک اندازه‌گیری

۳-۳-۲-۲ لایه‌های انتزاعی سخت‌افزار (PyVISA و Pymeasure)

برای ساده‌سازی فرآیند ارتباط با تجهیزات، از دو کتابخانه در پایتون استفاده شد. کتابخانه Pymeasure یک لایه انتزاعی شیء‌گرا بر روی دستورات متنی تجهیزات فراهم می‌کند. برای دستگاه‌های مورد استفاده در این پروژه، کلاس‌های درایور هر دستگاه توسعه داده شدند تا تمام قابلیت‌های مورد نیاز را پوشش دهند.

فصل سوم: روش پیشنهادی و نتیجه‌گیری

همانطور که در دیاگرام پایین مشاهده می‌شود، دو کلاس HP3458A و HP53131A به ترتیب برای دستگاه‌های مولتی‌متر و شمارنده توسعه داده شدند. این دو کلاس، کلاس داخلی کتابخانه Pymeasure را ارث‌بری می‌کنند. هر یک از متدها یک اندازه‌گیری مشخص را روی دستگاه پیکربندی می‌کنند که به عنوان نمونه در شکل نمایش داده شده است. این کار باعث شد تا لایه‌های بالاتر به جای کار با دستورات رشته‌ای، با متدهای معنادار کار کنند.

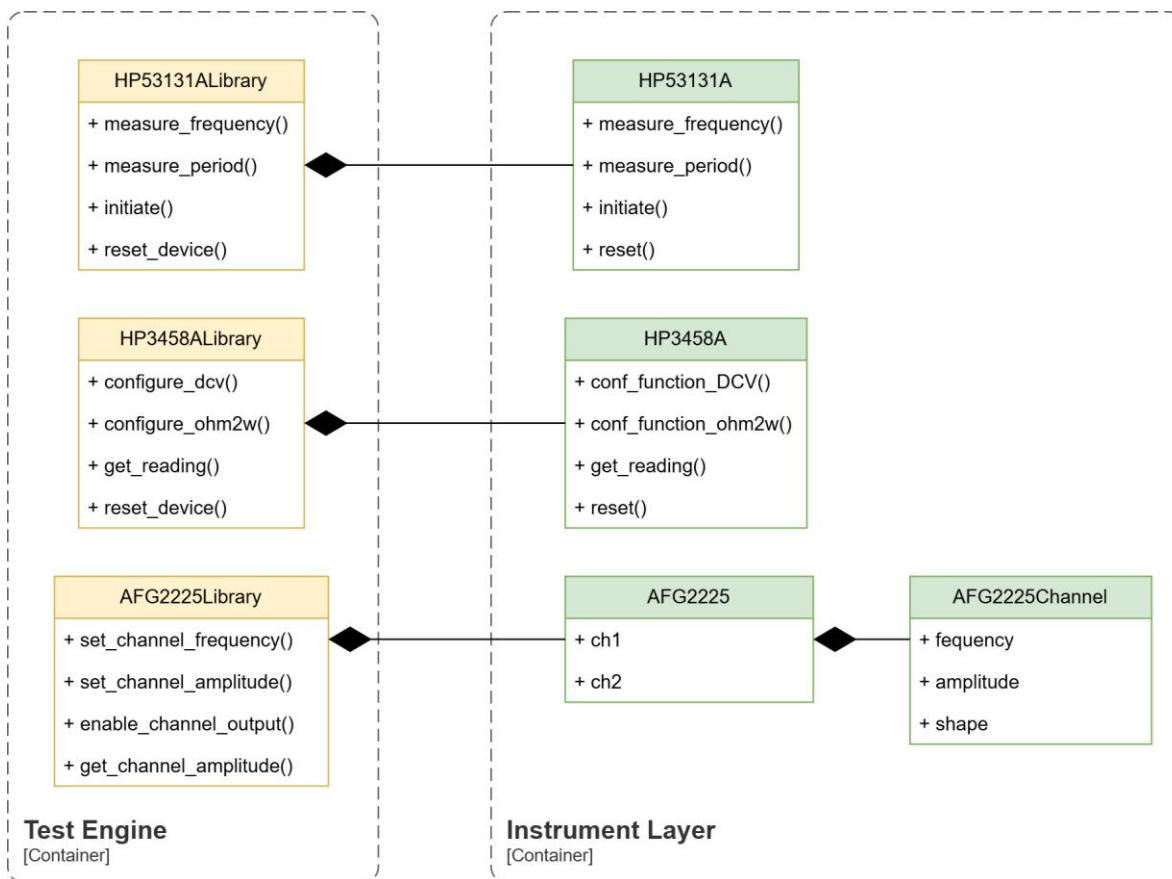


شکل ۲۸ نمای کد لایه انتزاعی سخت‌افزار

در لایه پایین‌تر، Pymeasure از PyVISA برای مدیریت ارتباطات فیزیکی استفاده می‌کند. PyVISA جزئیات پیچیده پروتکل‌های ارتباطی مانند TCP/IP, USB و GPIB را از دید توسعه‌دهنده پنهان می‌کند.

۳-۳-۲-۳ ارتباط کتابخانه‌های Robot با لایه‌های انتزاعی سخت‌افزار

در لایه‌های انتزاعی سخت‌افزار ما به ازای هر یک از دستگاه‌ها یک کلاس درایور به همراه متدهای معنادار توسعه دادیم که کار با دستورات رشته‌ای SCPI را مخفی می‌کند. حال در یک لایه بالاتر، لایه موتور تست، برای هر یک از کلاس‌های درایور و متدهای آن یک کلاس wrapper توسعه داده شد. این کلاس‌ها در مفاهیم چارچوب Robot، کتابخانه و متدهای آن کلمات کلیدی^{۸۲} نام برده می‌شود. مهندس آزمون هنگام نوشتن تست، کتابخانه‌های هر یک از دستگاه‌های مورد استفاده را در فایل تست اضافه^{۸۳} می‌کند و کلمات کلیدی آن را فراخوانی می‌کند. در شکل ۲۵ مشاهده می‌شود که کتابخانه‌های دستگاه، یک شی از کلاس‌های کنترل سخت‌افزار را نگهداری می‌کنند.

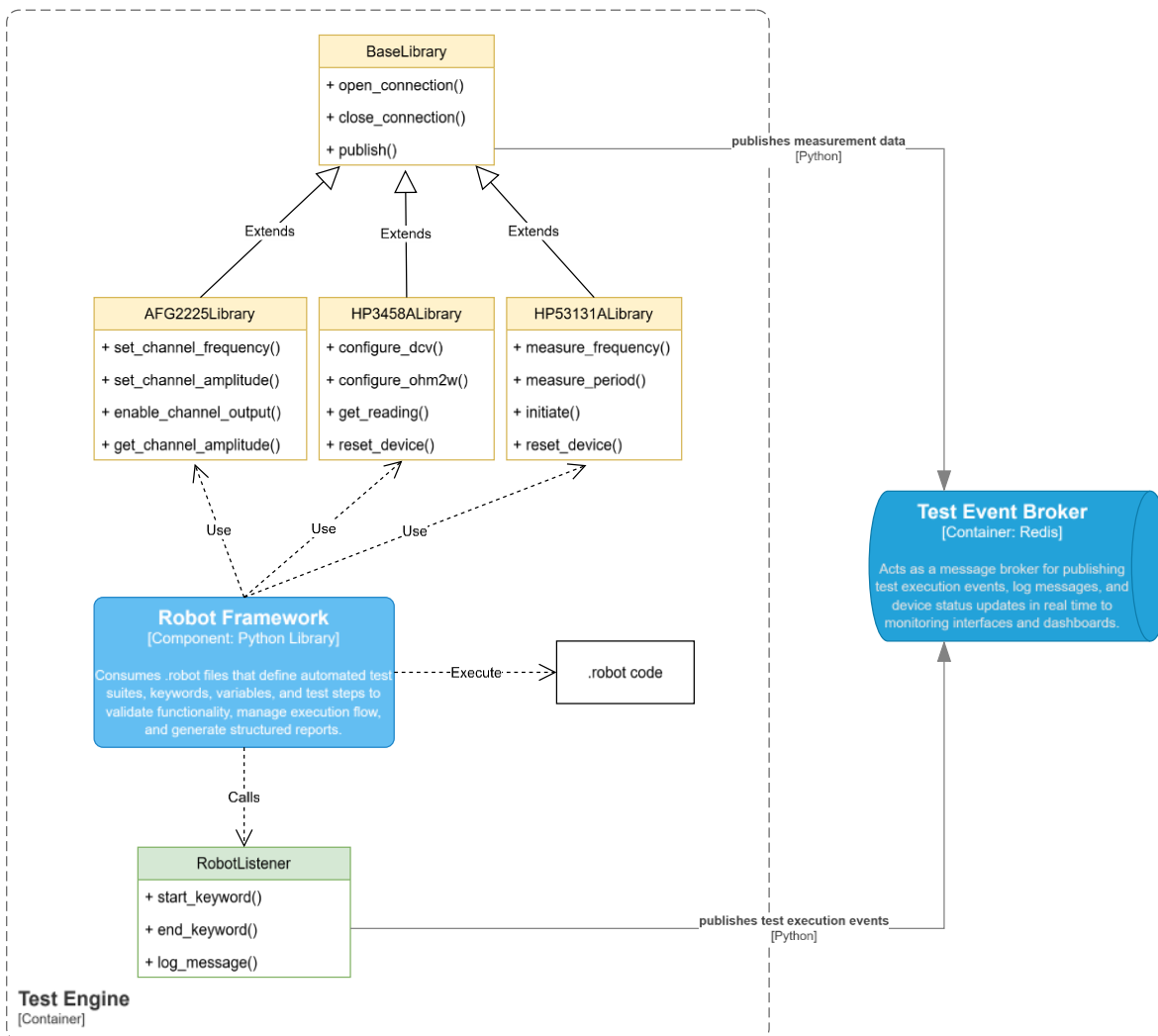


شکل ۲۹ نمودار کلاس لایه کنترل سخت‌افزار و لایه موتور تست

⁸² Keyword

⁸³ import

برای درک عمیق‌تر نحوه عملکرد موتور تست، ساختار داخلی آن را در سطح کلاس‌های پایتون بررسی می‌کنیم. نمودار شکل ۲۶ نشان می‌دهد که چگونه اجزای مختلف کد با یکدیگر در ارتباط هستند.



شکل ۳۰ نمودار کلاس ارتباط هسته فریمورک Robot با کلاس‌ها و containerها

کتابخانه‌های تجهیزات از کلاس پایه **BaseLibrary** به منظور جلوگیری از تکرار کد و ایجاد یک ساختار یکپارچه، ارث‌بری می‌کنند. این کلاس والد، مسئولیت‌های مشترک زیر را به هر یک از کلاس‌های فرزند اضافه می‌کند:

فصل سوم: روش پیشنهادی و نتیجه‌گیری

- برقراری و قطع ارتباط با تجهیزات.
 - پس از هر اندازه‌گیری، با فراخوانی شدن متد publish داده‌های دریافتی در صف پیام منتشر می‌شوند تا توسط رابط کاربری نمایش داده شود.
- کلاس RobotListener به رویدادهای داخلی Robot Framework متصل می‌شود. وظیفه اصلی آن، اطلاع‌رسانی در مورد وضعیت اجرای تست است. این کلاس با پیاده‌سازی متدهایی مانند start_keyword و end_keyword، رویدادهای مربوط به شروع و پایان هر گام از تست را در صف پیام منتشر می‌کند. این اطلاعات به رابط کاربری اجازه می‌دهد تا پیشرفت اجرای تست را به کاربر نشان دهد.

۵-۲-۳ مبدل پروتکل GPIB با Raspberry Pi

در این پروژه ابتدا از مبدل Prologix استفاده شد. ولی به خاطر گران‌قیمت بودن و تهیه کردن سخت آن، تصمیم گرفته شد تا یک مبدل جدید به کمک برد رزبری پای و درایورهای متن‌باز لینوکس توسعه داده شود.

۱. معماری راهکار

ایده اصلی این راهکار، استفاده از پایه‌های ورودی/خروجی عمومی (GPIO) رزبری پای برای شبیه‌سازی نرم‌افزاری سیگنال‌های پروتکل GPIB است. این تکنیک که با نام Bit-Banging شناخته می‌شود، به ما اجازه می‌دهد تا بدون نیاز به سخت‌افزار تخصصی و تنها با کنترل دقیق نرم‌افزاری پین‌های GPIO، عملکرد یک کنترلر GPIB را شبیه‌سازی کنیم. این پیاده‌سازی به کمک مجموعه درایورهای Linux-GPIB انجام شد.

۲. معرفی درایور Linux-GPIB

Linux-GPIB یک پروژه متن‌باز برای سیستم‌عامل لینوکس است که پشتیبانی از انواع سخت‌افزارهای کنترلر GPIB را فراهم می‌کند. این درایور شامل اجزای زیر است:

- ماژول‌های کرنل: درایورهای سطح پایینی که مستقیماً با سخت‌افزار ارتباط برقرار می‌کنند. که یکی از ماژول‌های کرنل در این مجموعه، gpib_bitbang است. این درایور به طور خاص برای پیاده‌سازی کنترلر GPIB از طریق پین‌های GPIO طراحی شده است.

فصل سوم: روش پیشنهادی و نتیجه‌گیری

- کتابخانه سطح کاربر^{۸۴}: API‌هایی به زبان‌های Python و C ارائه می‌دهد که به برنامه‌های کاربر در سطح سیستم‌عامل اجازه‌ی خواندن و نوشتن را فراهم می‌کند.

۳. فرآیند پیاده‌سازی

ابتدا با جستجو میان پروژه‌های متن‌باز، شماتیک یک شیلد رزبری‌پای که پین‌های کانکتور ۲۴ پین GPIB را به پین‌های GPIO روی برد رزبری‌پای متصل می‌کرد را پیدا کردیم و یک نمونه از آن ساخته شد. سپس با دریافت سورس‌کد درایور linux-gpib، روی سیستم‌عامل و معماری مشخص رزبری‌پای کامپایل و نصب شد. در این مرحله مبدل پروتکل GPIB ما آماده برقراری اتصال به تجهیزات شد. در شکل ۲۷ نمایی از برد رزبری‌پای مشاهده می‌شود که یک شیلد به پین‌های GPIO متصل شده است و در شکل ۲۸ نمایی از نحوه اتصال آن به پشت دستگاه شمارنده نمایان است.



شکل ۳۱ عکس مبدل GPIB ساخته شده به کمک برد رزبری‌پای

⁸⁴ User space



شکل ۳۲ عکس اتصال مبدل رزبری پای به دستگاه شمارنده

برای این که این کنترلر مانند مبدل Prologix از راه دور قابل دسترس باشد، با نصب و اجرای کتابخانه پایتونی pyvisa-proxy یک سرور RPC روی رزبری پای به اجرا درآمد. زمانی که در سیستم کلاینت (مثلا یک لپ‌تاپ) یکی از توابع کتابخانه PyVISA فراخوانی شود (مثلا تابع write برای ارسال یک دستور) همین تابع با پارامترهای آن روی رزبری پای اجرا می‌شود.

۳-۳-۲-۶ پیاده‌سازی مبدل پروتکل CAN با رزبری پای

مشابه رویکرد پیاده‌سازی انجام شده برای مبدل GPIB، ابتدا یک شیلد سخت‌افزاری طراحی و ساخته شد که ماجول MCP2515 را بر روی خود داشت و از طریق پین‌های SPI به GPIOهای رزبری پای متصل می‌شد. برخلاف پروتکل GPIB که با تکنیک Bit-Banging شبیه‌سازی شد، برای پروتکل CAN از رابط SPI^{۸۵} موجود بر روی رزبری پای برای اتصال از یک ماجول کنترلر CAN به نام MCP2515 استفاده شد که این ماجول به طور کامل پروتکل CAN را در سخت‌افزار پیاده‌سازی کرده است. ارتباط با این شیلد از طریق ابزار SocketCAN در لینوکس انجام می‌شود. SocketCAN با نصب بسته can-utils^{۸۶} روی سیستم‌عامل رزبری پای نصب می‌شود. این ابزار رابط‌های CAN^{۸۷} را به صورت یک رابط شبکه مجازی (مانند can0) در سیستم‌عامل نمایش می‌دهد.

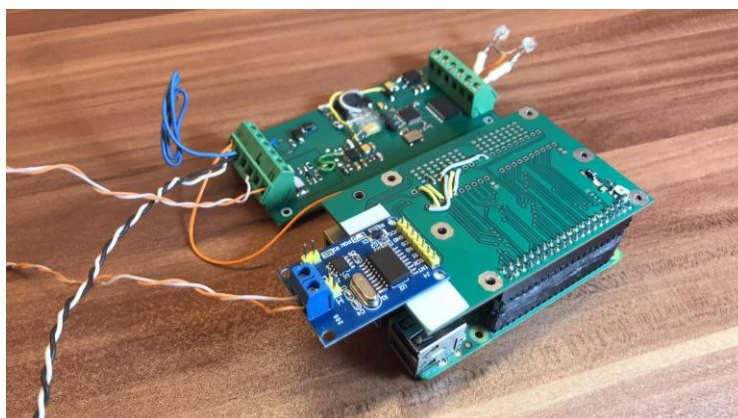
^{۸۵} Serial Peripheral Interface

^{۸۶} <https://github.com/linux-can/can-utils>

^{۸۷} Interface

فصل سوم: روش پیشنهادی و نتیجه‌گیری

از طریق محیط ترمینال، اتصال با یکی از دستگاه‌های تحت پروتکل CAN برقرار شد و با ارسال دستور و دریافت پاسخ، صحت برقراری ارتباط تضمین شد. بار دیگر با نصب کتابخانه python-can این ارتباط با دستگاه CAN از طریق یک برنامه پایتون صورت گرفت. برای آن که این کنترلر هم از طریق شبکه در دسترس باشد، کتابخانه ⁸⁸python-can-remote این امکان را می‌دهد تا با ایجاد یک سرور سوکت، رابط can0 را تحت شبکه در دسترس قرار دهد.



شکل ۳۳ عکس مایکول mcp2515 که برای تبدیل پین سریال SPI رزبری پای به پروتکل CAN مورد استفاده قرار گرفته است

۷-۲-۳-۳ افزودن یک متد جدید به PyVISA

درایور linux-gpib پایین‌ترین لایه نرم‌افزاری است که باس GPIB را مدیریت می‌کند و از طریق کتابخانه سطح کاربر خود به برنامه‌های روی سیستم عامل اجازه خواندن، نوشتن و اعلام وضعیت باس را می‌دهد. وضعیت باس می‌تواند شامل صفر و یک بودن خط‌های باس و handshake باشد.

هنگام مهاجرت از مبدل Prologix به مبدل رزبری پای و کتابخانه PyVISA متوجه یک محدودیت در کتابخانه PyVISA شدیم. این محدودیت ناتوانی این کتابخانه در گرفتن وضعیت باس از linux-gpib بود. برای همگام‌سازی و مدیریت وقفه‌ها بین نرم‌افزار و تجهیزات اندازه‌گیری نیاز به دانستن وضعیت خط SRQ داشتیم. خط SRQ در باس GPIB زمانی

⁸⁸ <https://github.com/christiansandberg/python-can-remote>

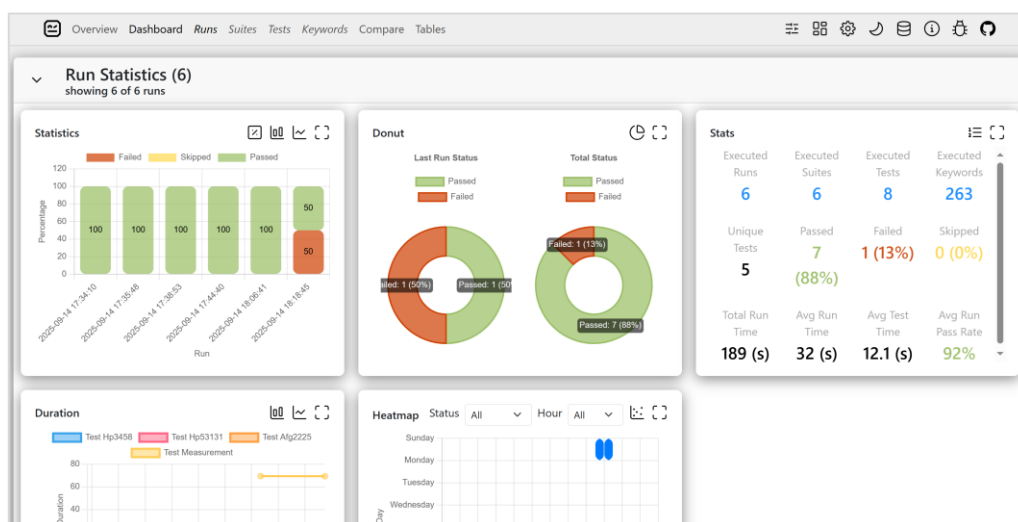
فصل سوم: روش پیشنهادی و نتیجه‌گیری

فعال می‌شود که یکی از دستگاه‌ها درخواست وقفه‌ای را صادر کند. خواندن وضعیت این خط به کمک مبدل Prologix انجام بود ولی در کتابخانه PyVISA پیاده‌سازی نشده بود.

برای رفع این محدودیت کتابخانه PyVISA را گسترش دادیم و یک متد جدید به نام `query_srq` به آن اضافه کردیم. این متد با فراخوانی سطح پایین توابع کتابخانه سطح کاربر `linux-gpib` وضعیت باس را دریافت می‌کند.

۸-۲-۳- تحلیل آماری نتایج اجرای تست

کتابخانه `robotframework-dashboard`⁸⁹ با دریافت داده‌ها و نتایج اجرای تست، تحلیل آماری انجام می‌دهد. این کتابخانه برای نگهداری تاریخچه نتایج تست از دیتابیس `sqlite` بهره می‌برد و یک محیط نموداری برای نمایش کیفیت اجرای تست‌ها در طول زمان به کاربر نمایش می‌دهد.



شکل ۳۴ محیط گرافیکی داشبورد تحلیل آماری Robot

⁸⁹ <https://github.com/timdegroot1996/robotframework-dashboard>

۳-۴ روش ارزیابی

در این بخش، روش پیشنهادی را از دو منظر ارزیابی و صحت‌سنجی می‌کنیم. ابتدا با محاسبه‌ی تأخیر زمانی ارسال دستور و دریافت آن شروع می‌کنیم.

۱-۴-۳ تأخیر

تأخیر به مدت زمانی که طول می‌کشد یک دستور برای دستگاه ارسال و یک پاسخ برای آن برگرداند، گفته می‌شود. برای محاسبه دقیق این تأخیر، دستوری باید انتخاب شود که زمان پردازش ناچیزی برای دستگاه داشته باشد (مثلاً یک دستور اندازه‌گیری ممکن است چند میلی‌ثانیه تا چند ثانیه طول بکشد). دستوری که در این ارزیابی استفاده خواهد شد IDN* است. این دستور کمترین بار پردازشی را برای دستگاه دارد و دستگاه در جواب یک رشته شامل نام و مدل دستگاه را بر می‌گرداند. به این صورت فقط تأخیر لایه‌های نرم‌افزار و شبکه محاسبه می‌شود.

در جدول پایین نتیجه ارزیابی قابل مشاهده است. اتصال بین لایه موتور تست با مبدل رزبری پای تحت شبکه است، به همین خاطر یک بار هم دستور به طور مستقیم از روی مبدل رزبری برای دستگاه ارسال و تأخیر آن محاسبه شد. به دلیل در دسترس نبودن دستگاه اندازه‌گیر توان، تأخیر برای این دستگاه محاسبه نشد.

جدول ۷ مقایسه تأخیرهای ارسال دستور IDN* به دستگاه‌ها از لایه‌های مختلف

تأخیر مستقیم از لایه مبدل رزبری پای	تأخیر از لایه موتور تست Robot			دستگاه
	با مبدل رزبری پای	با مبدل Prologix	اتصال مستقیم به لپ‌تاپ	
۰.۰۰۳۲	۰.۰۲	۰.۰۳۰۷	-	مولتی‌متر - HP3458
۰.۰۰۲۵	۰.۰۲	۰.۰۳۱۲	-	شمارنده - HP53131
-	-	-	۰.۰۳	Function generator
-	-	-	-	Power meter

۲-۴-۳ صحت‌سنجی اندازه‌گیری

در این بخش به کمک موتور تست، یک سناریو اندازه‌گیری برای دستگاه‌های مولتی‌متر و شمارنده اجرا خواهیم کرد. در این آزمون یک مقدار اندازه‌گیری از هر یک از دستگاه‌ها خوانده می‌شود و انتظار می‌رود با سیگنال ورودی داده شده مطابقت داشته باشد. در قسمت پایین قطعه کد به زبان Robot با دو تست‌کیس آورده شده است. در تست اول، دستگاه مولتی‌متر باید یک مقدار بین ۹.۹ تا ۱۰.۱ ولت را اندازه‌گیری کند. در تست دوم، دستگاه تولیدکننده سیگنال ابتدا تنظیم می‌شود تا یک فرکانس ۱۰۰ کیلوهرتز تولید کند و سپس دستگاه شمارنده آن را باید اندازه‌گیری کند.

```
*** Settings ***
Library    HP3458ALibrary.py    AS    Multimeter
Library    HP53131ALibrary.py    AS    Counter
Library    AFG2225Library.py    AS    Function

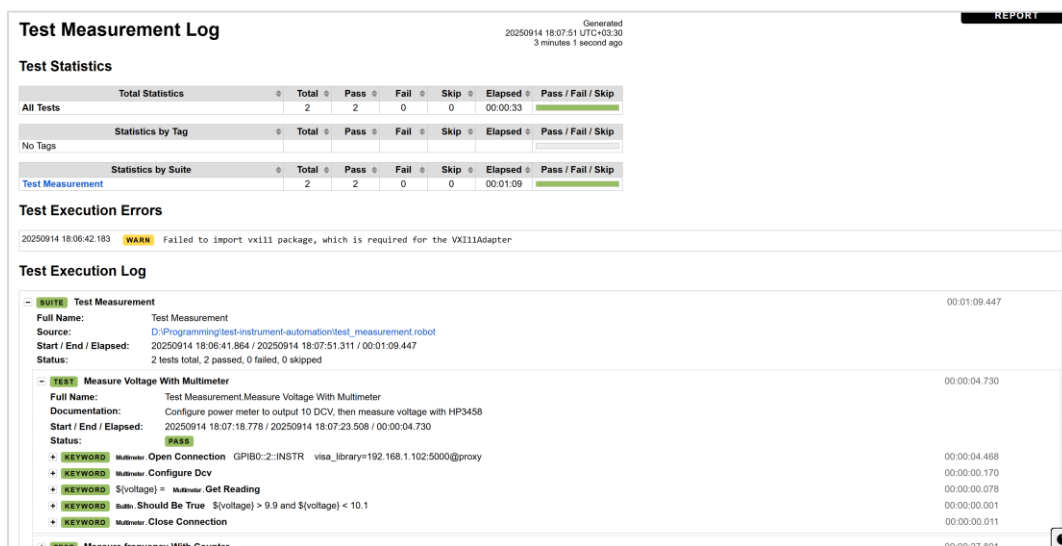
*** Keywords ***
Configure Function Generator
    [Arguments]    ${freq}    ${amp}
    Set Channel Frequency    1    ${freq}
    Set Channel Amplitude    1    ${amp}
    Set Channel Shape    1    sine
    Enable Channel Output    1
    Log    Configured AFG2225: ${freq} Hz, ${amp} Vpp

*** Test Cases ***
Measure Voltage With Multimeter
    [Documentation]    Configure power meter to output 10 DCV, then measure voltage with HP3458
    Multimeter.Open Connection    GPIB0::2::INSTR    visa_library=192.168.1.102:5000@proxy
    Configure Dcv
    ${voltage}=    Get Reading
    Should Be True    ${voltage} > 9.9 and ${voltage} < 10.1
    Multimeter.Close Connection

Measure frequency With Counter
    Counter.Open Connection    GPIB0::3::INSTR    visa_library=192.168.1.102:5000@proxy
    Function.Open Connection    ASRL5::INSTR
    Configure Function Generator    100_000    1.0
    Measure Frequency    channel=1
    ${freq}=    Initiate Wait And Fetch
    Should Be True    ${freq} > 99_990 and ${freq} < 100_010
    Function.Close Connection
    Counter.Close Connection
```

اجرای تست بالا با موفقیت انجام شد و در شکل ۳۲ نمونه‌ای از گزارش تولید قابل مشاهده است.

فصل سوم: روش پیشنهادی و نتیجه گیری



شکل ۳۵ گزارش تولید شده پس از اجرای یک نمونه تست

۳-۴-۳ مجوزها

تمام کتابخانه‌های استفاده شده در این پروژه متن‌باز بودند و در این بخش مجوزهای هر یک از این کتابخانه‌ها به همراه

لینک متن مجوز در جدول ۸ گردآوری شده است:

جدول ۸ لیست کتابخانه‌ها و مجوزهای هر یک از کتابخانه‌ها

نام کتابخانه	مجوز
Pymasure	MIT License
PyVISA	MIT License
Robot Framework	Apache License 2.0
Pyvisa-proxy	MIT License
Streamlit	Apache License 2.0
Redis	AGPLv3

۳-۵ نتایج

۱. تحلیل نتایج تاخیر

نتایج به وضوح نشان می‌دهد که مبدل سفارشی رزبری پای در ارتباط مستقیم با تجهیزات، عملکرد بسیار سریعی (بین ۲.۵ تا ۳.۲ میلی‌ثانیه) دارد که نشان‌دهنده کارایی بالای درایور linux-gpib است. هنگامی که دستورات از طریق موتور تست و بر بستر شبکه ارسال می‌شوند، تاخیر به حدود ۲۰ تا ۳۰ میلی‌ثانیه افزایش می‌یابد. این افزایش ناشی از ارسال دستورات تحت شبکه و لایه پروکسی است. با مقایسه تاخیر مبدل رزبری پای با مبدل تجاری Prologix متوجه نزدیک بودن تاخیر این دو مبدل به یکدیگر می‌شویم. از آنجایی که مبدل رزبری پای توانسته تاخیری مشابه Prologix ارائه دهد یک مزیت مهم برای این سخت‌افزار ارزان قیمت‌تر نسبت به Prologix است.

۲. تحلیل سناریوهای آزمون

در آزمون اول، سیستم توانست با موفقیت مولتی‌متر را پیکربندی کرده و یک مقدار ولتاژ را با دقت بالا اندازه‌گیری و اعتبارسنجی کند. آزمون دوم، یک سناریوی پیچیده‌تر و واقعی‌تر را شبیه‌سازی می‌کند که در آن سیستم باید دو دستگاه را هماهنگ کند. ابتدا دستگاه تولیدکننده سیگنال را برای تولید یک سیگنال ۱۰۰ کیلوهرتز تنظیم کرده و سپس با استفاده از دستگاه شمارنده، فرکانس همان سیگنال را اندازه‌گیری و در بازه خطای ۰.۰۱٪ تایید می‌کند. این نتایج نشان می‌دهد که چارچوب پیاده‌سازی‌شده در سطح ارتباطی پایدار است و در سطح عملکردی و هماهنگی چندین دستگاه نیز قابل اطمینان بوده و قادر به اجرای آزمون‌های دقیق و پیچیده صنعتی است.

۳-۶ جمع‌بندی

در این فصل، روش پیشنهادی برای ساخت یک چارچوب آزمون خودکار به همراه جزئیات کامل معماری و پیاده‌سازی آن تشریح گردید. ابتدا، به کمک مدل C4، یک نمای چندلایه از معماری سیستم ارائه شد که از بالاترین سطح تعاملات کاربر تا پایین‌ترین سطح اجزای نرم‌افزاری را پوشش می‌داد. این نمودار، اجزا و ارتباط زیرسیستم‌های مختلف را نمایش می‌داد. در ادامه، جزئیات پیاده‌سازی هر یک از اجزای اصلی معماری، از جمله رابط کاربری گرافیکی مبتنی بر Streamlit، موتور تست مبتنی بر Robot Framework و لایه‌های انتزاعی سخت‌افزار مورد بررسی قرار گرفت. انتخاب هر یک از این فناوری‌ها

فصل سوم: روش پیشنهادی و نتیجه‌گیری

با هدف دستیابی به توسعه سریع، انعطاف‌پذیری و استفاده از ابزارهای متن‌باز صورت پذیرفت. یکی از دستاوردها که در این فصل توضیح داده شد، پیاده‌سازی یک مبدل سخت‌افزاری سفارشی برای پروتکل GPIB و CAN با استفاده از رزبری پای بود که به عنوان یک راهکار برای غلبه بر محدودیت‌های هزینه مبدل‌های تجاری ارائه شد.

- [1] Agilnet Technilogies, *Agilent 53131A/132A 225 MHz Universal Counter - Operating guide*. [Online]. Available: https://dfzk-www.oss-cn-beijing.aliyuncs.com/www-PRD/resources/files/Ag_53131A-53132A_OperGuide_en.pdf
- [2] Agilnet Technilogies, *Agilent 53131A/132A 225 MHz Universal Counter - Programming Guide*. [Online]. Available: https://fy.chalmers.se/~f7x1h/elmatB/Instrument/53131_Prog.pdf
- [3] Agilnet Technilogies, *Agilent Technologies E4418B/E4419B Power Meters - Programming guide*. 2007.
- [4] Agilnet Technilogies, *Agilent Technologies E4418B/E4419B Power Meters - User guide*. [Online]. Available: https://dfzk-www.oss-cn-beijing.aliyuncs.com/www-PRD/resources/files/Ag_E4419B_UserGuide_en.pdf
- [5] Agilnet Technilogies, *Agilnet Technologies 3458A Multimeter*, 4th ed. 2000. [Online]. Available: https://dfzk-www.oss-cn-beijing.aliyuncs.com/www-PRD/resources/files/Ag_3458A_UserGuide_en.pdf
- [6] GW INSTRUK, *Arbitrary Function Generator AFG-2225*. [Online]. Available: <https://www.gwinstek.com/en-global/products/downloadSeriesDownNew/5264/430>
- [7] coolshou, *coolshou/linux-gpib*. (Sept. 16, 2025). C. Accessed: Sept. 16, 2025. [Online]. Available: <https://github.com/coolshou/linux-gpib>
- [8] Hewlett-Packard Company, *HP 3458A Operating, Programming & Configuration*, 1st ed. 1988.
- [9] Keysight, *Keysight 3458A Multimeter - Quick Reference*, 3rd ed. 2017.
- [10] “pymeasure/pymeasure at master · pymeasure/pymeasure,” GitHub. Accessed: Sept. 16, 2025. [Online]. Available: <https://github.com/pymeasure/pymeasure/tree/master/pymeasure>
- [11] H. E. Grecco, M. C. Dartiaill, G. Thalhammer-Thurner, T. Bronger, and F. Bauer, *PyVISA: the Python instrumentation package*. (Apr. 2023). Python. doi: [10.21105/joss.05304](https://doi.org/10.21105/joss.05304).
- [12] *robotframework/robotframework*. (Sept. 16, 2025). Python. Robot Framework. Accessed: Sept. 16, 2025. [Online]. Available: <https://github.com/robotframework/robotframework>
- [13] “Streamlit • A faster way to build and share data apps.” Accessed: Sept. 16, 2025. [Online]. Available: <https://streamlit.io/>
- [14] “The gpib4pi - GPIB for Raspberry Pi shield,” Hackster.io. Accessed: Sept. 20, 2025. [Online]. Available: <https://www.hackster.io/lightside-instruments/the-gpib4pi-gpib-for-raspberry-pi-shield-4b3e9a>

پیوست

- [لینک مخزن گیت‌هاب](#)
- [لینک یادداشت‌های پروژه:](#) در طول زمان انجام پروژه روزانه در این فایل یادداشت‌برداری می‌شد.
- [لینک نمودارهای مدل C4](#)

Title of thesis

Abstract

With the rapid growth of the electronics industry, automation of measurement processes has become a key factor in ensuring quality, speed, and accuracy. Various commercial and open-source software solutions have been developed to address this challenge. However, commercial closed-source tools lack the flexibility to add custom features and often provide limited support for diverse hardware and communication protocols. On the other hand, existing open-source projects, while more flexible, generally lack the integration needed to cover all requirements such as remote instrument control, measurement automation, and a unified graphical user interface.

To address these limitations, this project presents a Python-based software framework with a layered, distributed architecture. The framework includes a web-based graphical user interface for user interaction and real-time equipment monitoring, a test execution engine built on Robot Framework for defining and automating measurements, and abstract software layers for instrument communication and control. Furthermore, a hardware converter for GPIB and CAN protocols was implemented using Raspberry Pi and open-source Linux drivers, enabling instrument access over the network and supporting a distributed architecture.

The final outcome is a fully integrated automated testing system that streamlines measurement definition, remote instrument control, test execution, and real-time result monitoring.



Shahid Beheshti University
Faculty of Computer Science and Engineering

Design of an Automated System for Industrial Test Equipment

By:
Mohammad-Moein Arabi

A THESIS SUBMITTED
FOR THE DEGREE OF
BACHELOR OF SCIENCE

Supervisor
Dr. Seyed-Hosein Attarzadeh-Niaki

September 2025

